

操作系统实验报告

(2019-2020 第二学期)

姓 名： 邓 梹
学 号： 011750230
班 级： 1617001
指导教师： 朱小军

南京航空航天大学

2020 年 6 月 27 日

final

笔记本: OS

创建时间: 2020/6/27 16:46

更新时间: 2020/6/27 18:33

作者: 1300204274@qq.com

proj0

任务一 添加用户命令print30

1、任务描述

在xv6下，执行`ls`，会显示已经实现的命令列表。你需要添加一个新的命令：`printxy`，其中xy指学号最后两位，比如尾号为31的同学应当实现的命令是`print31`。当用户在界面敲入`printxy` Z后，屏幕会显示OS Lab 学号: Z。

例如，学号为041221135的同学应当实现命令`print35`，他的程序在输入 `print35 hello world`后，屏幕应当显示：

OS Lab 041221135: hello world

如果用户只输入`printxy`，没加其他参数，则屏幕在显示完“OS Lab 学号:”后，应当换行。

2、实现方法

在 `proj0-revise` 目录下创建 `print30.c` 文件，其内容如下：

```
#include "types.h"
#include "stat.h"
#include "user.h"
int main(int argc, char* argv[])
{
    printf(1, "OS Lab 011750230:");
    if (argc != 1) {
        for (int i = 1; i < argc; i++) {
```

```

        printf(l, "%s", argv[i]);
        if (i < argc - 1) {
            printf(l, " ");
        }
    }
    printf(l, "\n");
    exit();
}

```

此时进入 xv6，执行`ls` 指令，可以看到还没有`print30` 命令。

要使用`print30` 命令，还需要在 Makefile 文件中的 UPROGS 中加入`_print30`,使系统知道有这么个用户命令。以后用户使用 `print30` 命令时，系统则调用`print30.c` 文件中的 `main` 函数进行处理。

3、实验结果

修改完成后执行效果如下：

```

xv6...
cpu0: starting 0
sb: size 1000 nblocks 941 ninodes 200 nlog 30 logstart 2 inodestart 32 bmap start 58
init: starting sh
$ print30
main-loop: WARNING: I/O thread spun for 1000 iterations
OS Lab 011750230:
$ print30 hello world
OS Lab 011750230:print30 hello world
$ print30 你好
OS Lab 011750230:print30 你好

```

任务二 添加内核输出语句

1、任务描述

在系统启动时，在内核打印"cpu0: starting 0"之后，打印"sb: size 1000 nblocks"之前，插入一行，打印自己的用户名和学号，最后显示效果如下

```

xv6...
cpu0: starting 0
zhangsan 041221135
sb: size 1000 nblocks 941 ninodes 200 nlog 30 logstart 2 inodestart 32 bmap
start 58

```

```
init: starting sh
$
```

2、实现方法

处理思路：搜索关键字“starting”，找到打印“cpu0: starting 0”的语句，那就在它后面打印我想打印的信息。

在 `main.c` 文件中的 `static void mpmain(void)` 函数中，

在语句 `cprintf("cpu%d: starting %d\n", cpuid(), cpuid());`

之后添加语句 `cprintf("dengxiao 011750230\n");` 即可。

这里是在内核态下输出。

3、实验结果

```
xv6...
cpu0: starting 0
dengxiao 011750230
sb: size 1000 nblocks 941 ninodes 200 nlog 30 logstart 2 inodestart 32 bmap start 58
init: starting sh
```

在内核输出和在用户空间输出的区别：

- 内核状态下执行的代码在内核空间中，用户状态下执行的代码不在内核空间中。
- 执行内核代码时CPU处在权限更高的状态。
- 内核状态可以访问用户空间的内容，反之则不行。

任务三 添加系统调用shutdown

1、任务描述

- xv6操作系统默认没有关机命令，你需要添加用户命令 `shutdown`。
- 在xv6中，以下语句可以让它关机：`outw(0x604,0x2000)`。但由于这个语句使用了特权指令，需要在内核态执行，所以不能直接在用户程序中使用。
- 你需要实现一个系统调用 `void shutdown (void)`。在系统调用中使用 `outw(0x604,0x2000)` 令机器关机，然后在用户命令中调用你实现的这个系统调用。

判断是否成功的方法是：输入 `shutdown` 后，退出 `qemu` 到 `ssh` 窗口。

2、实现方法

- 在 `user.h` 和 `defs.h` 中声明 `shutdown` 函数 `int shutdown(void)`; 并在 `proc.c` 文件中实现具体的系统调用函数功能。其中的22代表系统调用号，在后续操作中，`sys_shutdown`的系统调用号即为22。为什么是22呢？因为已经用到了21，那连续一点，咱就用22。目前，它还不是一个系统调用函数。以后，22将作为参数，使中断时直到使用哪个系统调用。

```
int shutdown(void) {
    outw(0x604, 0x2000);
    return 22;
}
```

- 在 `usys.S` 的最后加上 `SYSCALL(shutdown)`。它是宏定义，将被替换为调用系统调用的汇编代码。
- 加入系统调用号的定义 `#define SYS_shutdown 22`（在 `sys_call.h` 的最后）。这里需要注意的是函数名(`shutdown`)和系统调用标识符(`SYS_shutdown`)不能同名。
- 在 `sys_call.c` 中注册该函数。首先声明外部函数 `extern int sys_shutdown(void)`; 然后在 `static int (*syscalls[])(void) = { }` 中添加 `"[SYS_shutdown] sys_shutdown, "`。目的是将系统调用标识 `SYS_shutdown` 和系统调用函数 `sys_shutdown` 关联起来。
- 最后一步需要把系统调用 `sys_shutdown` 和功能函数 `shutdown` 关联起来。在 `sysproc.c` 中，添加对于系统调用 `sys_shutdown` 函数的实现,他调用 `shutdown()`。

```
int sys_shutdown(void) {
    return shutdown();
}
```

- 最后，在系统中添加 `shutdown` 命令。 `shutdown.c` 文件内容如下：

```
#include "types.h"
#include "stat.h"
#include "user.h"

int main(int argc, char *argv[])
{
    if(argc!=1) {
        printf(1, "Please do not input arguments with 'shutdown'\n");
    }
    else{
        shutdown();
    }
}
```

```
}  
    exit();  
}
```

3、实验结果

```
xv6...  
cpu0: starting 0  
dengxiao 011750230  
sb: size 1000 nblocks 941 ninodes 200 nlog 30 logstart 2 inodestart 32 bmap start 58  
init: starting sh  
$ print30 hello world!  
OS Lab 011750230: hello world!  
$ print30  
OS Lab 011750230:  
$ shutdown  
dengxiao@VM-199-186-ubuntu:~/proj0-revise$
```

[参考资料]

[1] xv6操作系统中添加用户应用程序

<https://blog.csdn.net/yyd19981117/article/details/95199634>

[2] xv6操作系统中增加一个系统调用

<https://blog.csdn.net/yyd19981117/article/details/95200267>

proj1

1. 带参数的系统调用

一、任务描述：

proj0中添加的系统调用不带任何参数，你需要修改系统调用为：

```
int shutdown(int a);
```

其中a为传入的参数。同时修改shutdown.c文件，当用户在命令行输入 `shutdown 20` 时，屏幕上打印一行字后退出，打印的字符串为：

```
Leaving with code 20.
```

发布的源码中已包含 `shutdown` 的命令以及原始的内核实现，你只需要修改这个实现即可。

二、 实现方法

1. 修改 shutdown.c，把参数传过去

```
int
main(int argc, char *argv[])
{
    shutdown(atoi(argv[1]));
    exit();
}
```

2. 在 sysproc.c 中修改 sys_shutdown(void)函数，获取参数并输出相应信息。

```
int
sys_shutdown(void){
    int n;
    if(argint(0, &n) < 0)
        return -1;
    cprintf("Leaving with code %d.\n", n);
    outw(0x604, 0x2000);
    return 0;
}
```

三、 运行结果

```
xv6...
cpu0: starting 0
sb: size 1000 nblocks 941 ninodes 200 nlog 30 logstart 2 inodestart 32 bmap start 58
init: starting sh
$ shutdown 30
Leaving with code 30.
dengxiao@VM-199-186-ubuntu:~/proj1-revise$
```

问题：

为何 `argint` 推算出的第 n 个参数 (n 从 0 开始) 位置是在 `esp+4+4*n`，具体回答，4 是什么意思，为什么不是 8，不是 16？为什么不是 `esp+4*n`？

答：

4 是 `int` 类型的大小，单位是字节。(因为是 4，所以不是 8，也不是 16。)

`esp` 是新栈顶，它所指向的位置还没有数据，因为参数下标从 0 开始，所以是 `esp+4+4*n`，而不是 `esp+4*n`。(不是 `esp-4-4*n` 是因为栈从高地址向低地址生长)。

2. 理解进程切换

问题 1： CPU 在执行 `scheduler()` 时运行在用户态还是内核态？运行在哪个栈上面？

```
dengxiao@VM-199-186-ubuntu: ~/proj1-base
#0 popcli () at spinlock.c:121
#1 0x80103add in scheduler () at proc.c:353
#2 0x80102e3f in mpmain () at main.c:57
#3 0x80102f7f in main () at main.c:37
(gdb) info res
Undefined info command: "res". Try "help info".
(gdb) info reg
eax            0x200      512
ecx            0x1        1
edx            0x0        0
ebx            0x80114c54   -2146349996
esp            0x8010b540   0x8010b540 <stack+3968>
ebp            0x8010b548   0x8010b548 <stack+3976>
esi            0x80112780   -2146359424
edi            0x80112784   -2146359420
eip            0x801043a0   0x801043a0 <popcli+80>
eflags        0x206      [ PF IF ]
cs             0x8        8
ss             0x10       16
ds             0x10       16
es             0x10       16
fs             0x0        0
gs             0x0        0
(gdb)
```

答：此时 $cs = 0x8 = 1000B$ ， 所以 `cpu` 执行 `scheduler()` 时运行在内核态，
 $esp = 0x8010\ b540$ ，运行在内核的栈里。

问题 2： 当你在命令行敲下 `shutdown` 时，系统会创建一个进程执行 `shutdown.c` 中的代码，当 CPU 执行以下三条指令时 `movl $SYS_shutdown, %eax; int $T_SYSCALL; ret` 时，CPU 运行在用户态还是内核态？运行在哪个栈上面？

答：

```
0x00000026 in ?? ()
(gdb)
=> 0x2b:      call    0xd62
0x0000002b in ?? ()
(gdb)
=> 0xd62:      mov     $0xf,%eax
0x00000d62 in ?? ()
(gdb) info reg
eax            0x0      0
ecx            0x3fec   16364
edx            0x0      0
ebx            0x0      0
esp            0x3fbc   0x3fbc
ebp            0x3fd8   0x3fd8
esi            0x0      0
edi            0x0      0
eip            0xd62    0xd62
eflags         0x212    [ AF IF ]
cs             0x1b     27
ss             0x23     35
ds             0x23     35
es             0x23     35
fs             0x0      0
gs             0x0      0
(gdb) si
=> 0xd67:      int     $0x40
0x00000d67 in ?? ()
(gdb) info reg
eax            0xf      15
ecx            0x3fec   16364
edx            0x0      0
ebx            0x0      0
esp            0x3fbc   0x3fbc
ebp            0x3fd8   0x3fd8
esi            0x0      0
edi            0x0      0
eip            0xd67    0xd67
eflags         0x212    [ AF IF ]
cs             0x1b     27
ss             0x23     35
ds             0x23     35
es             0x23     35
fs             0x0      0
gs             0x0      0
(gdb) si
=> 0x80105da9 <vector64+2>:      push    $0x40
vector64 () at vectors.S:320
320      pushl $64
(gdb) info reg
eax            0xf      15
ecx            0x3fec   16364
edx            0x0      0
ebx            0x0      0
esp            0x8dffefe8      0x8dffefe8
ebp            0x3fd8   0x3fd8
esi            0x0      0
edi            0x0      0
eip            0x80105da9      0x80105da9 <vector64+2>
eflags         0x212    [ AF IF ]
cs             0x8      8
ss             0x10     16
ds             0x23     35
es             0x23     35
fs             0x0      0
gs             0x0      0
(gdb) si
=> 0x80105dab <vector64+4>:      jmp     0x801056b7 <alltraps>
vector64 () at vectors.S:321
```

执行 `movl $SYS_shutdown, %eax;` 之后, `cs = 0x1b`, 此时 CPU 在用户态, 运行在用户栈上。

执行 `int $T_SYSCALL`; 之后, `cs = 0x8`, `esp = 0x8dffefe8`, 此时 CPU 在用户态, 运行在进程的用户栈上。

执行 `ret` 时, CPU 是在内核态, 运行在进程的内核栈上。

问题 3: 在执行命令 `shutdown` 的过程中, 当 `cpu` 执行到涉及特权指令的函数 `outw()` 时, CPU 运行在用户态还是内核态? 运行在哪个栈上面?

```
dengxiao@VM-199-186-ubuntu: ~/proj1-base
99      outw(0x604, 0x2000);
(gdb) si
=> 0x8010563d <sys_shutdown+45>:      mov     $0x2000,%eax
0x8010563d in outw (data=8192, port=1540) at x86.h:30
30      asm volatile("out %0,%1" : : "a" (data), "d" (port));
(gdb) bt
#0  0x8010563d in outw (data=8192, port=1540) at x86.h:30
#1  sys_shutdown () at sysproc.c:99
#2  0x80104837 in syscall () at syscall.c:141
#3  0x80105909 in trap (tf=0x8dfbefb4) at trap.c:43
#4  0x8010566c in alltraps () at trapasm.S:20
#5  0x8dfbefb4 in ?? ()
Backtrace stopped: previous frame inner to this frame (corrupt stack?)
(gdb) info reg
eax                0x200      512
ecx                0x1         1
edx                0x604      1540
ebx                0x80112e4c   -2146357684
esp                0x8dfbef40   0x8dfbef40
ebp                0x8dfbef68   0x8dfbef68
esi                0x8dfbefb4   -1912868940
edi                0x8dfbefb4   -1912868940
eip                0x8010563d   0x8010563d <sys_shutdown+45>
eflags             0x282      [ SF IF ]
cs                 0x8         8
ss                 0x10        16
ds                 0x10        16
es                 0x10        16
fs                 0x0         0
gs                 0x0         0
(gdb)
```

答: 此时 `cs=0x8 = 1000B`, 所以 `cpu` 执行 `outw()` 时运行在内核态, `esp = 0x8dfbef40`, 运行在进程的内核栈里。

问题 4: 为何在执行完 `swtch` 函数后, `cpu` 没有像普通函数调用一样返回到 `scheduler` 函数中?

答:

```

(gdb) si
=> 0x80104679 <swtch+14>:      mov    %edx,%esp
22      movl %edx, %esp
(gdb) bt
#0  swtch () at swtch.S:22
#1  0x80112784 in cpus ()
#2  0x80112780 in ?? ()
#3  0x80102e3f in mpmain () at main.c:57
#4  0x80102f7f in main () at main.c:37
(gdb) si
=> 0x8010467b <swtch+16>:      pop    %edi
swtch () at swtch.S:25
25      popl %edi
(gdb) bt
#0  swtch () at swtch.S:25
#1  0x00000000 in ?? ()
Backtrace stopped: previous frame inner to this frame (corrupt stack)
(gdb) |

```

在执行代码 `mov %edx, %esp` (把 `edx` 的内容赋给 `esp`) 之后, 就出现了“Backtrace stopped: previous frame inner to this frame (corrupt stack?)”。

此时 CPU 更换了运行的栈, 运行了别的进程, 而不是简单的返回调用 `scheduler()` 的位置。

3. 子进程优先的 fork

一、 任务描述

阅读proc.c中的int fork(void)函数。fork涉及到两个进程，新的进程称为子进程，原始的进程称为父进程。xv6执行完fork后，父进程先运行，然后子进程运行。你需要修改这个行为，让子进程优先运行。

实现以下系统调用，以控制是否启用你的算法。此系统调用已经分配了系统调用号，你只需要在内核中实现即可。

```
void fork_winner(int winner);
```

- 若winner=1，表示启用你的算法；若winner=0，表示不启用你的算法。
- 默认情况下不启用你的算法。
- 如果你的实现是正确的，运行forktest命令后你应当能观察到，设置child为winner时，大多数情况是先打印child后打印parent；设置parent为winner时，大多数情况是先打印parent后打印child。
- 不允许修改forktest.c文件（即使编程风格不好，也请不要修改）。

二、实现方法

1. 在 proc.c 中设置全局变量 whoWin，whoWin = 1 时子进程优先，否则父进程优先。其初始值是 0。

并实现 `void fork_winner(int winner)` 函数，用于改变 whoWin 的状态。

```
int whoWin = 0;
void fork_winner(int winner){
    whoWin = winner;
}
```

2. 在 defs.h 中加入 `void fork_winner(int winner)` 声明，因为 sysproc.c 中要调用 `void fork_winner(int winner)` 函数。
3. 在 sysproc.c 中实现 `int sys_fork_winner(void)` 如下，

```
int sys_fork_winner(void){
    int winner;
    if(argint(0, &winner) < 0)
        return -1;
    fork_winner(winner);
    return 23;
}
```

4. 在 proc.c 中的 fork() 返回 pid 之前，添加语句，

```
if(whoWin == 1)    yield();
```

yield() 函数的功能时使本进程（线程）主动放弃占用 CPU。

三、运行结果：

```
$ forktest
Fork test
Set child as winner
Trial 0: child! parent!
Trial 1: child! parent!
Trial 2: child! parent!
Trial 3: child! parent!
Trial 4: child! parent!
Trial 5: child! parent!
Trial 6: child! parent!
Trial 7: child! parent!
Trial 8: child! parent!
Trial 9: child! parent!
```

```
Set parent as winner
Trial 0: parent! child!
Trial 1: parent! child!
Trial 2: parent! child!
Trial 3: parent! child!
Trial 4: child! parent!
Trial 5: parent! child!
Trial 6: parent! child!
Trial 7: parent! child!
Trial 8: parent! child!
Trial 9: parent! child!
```

```
$ █
```

```
$ forktest
Fork test
Set child as winner
Trial 0: child! parent!
Trial 1: child! parent!
Trial 2: child! parent!
Trial 3: child! parent!
Trial 4: child! parent!
Trial 5: child! parent!
Trial 6: child! parent!
Trial 7: child! parent!
Trial 8: child parent! !
Trial 9: child! parent!
```

```
Set parent as winner
Trial 0: parent! child!
Trial 1: parent! child!
Trial 2: parent! child!
Trial 3: parent! child!
Trial 4: parent! child!
Trial 5: parent! child!
Trial 6: parent! child!
Trial 7: par child! ent!
Trial 8: parent! child!
Trial 9: parent! child!
```

```
$ █
```

```
$ forktest
Fork test
Set child as winner
Trial 0: parent! child!
Trial 1: child! parent!
Trial 2: child! parent!
Trial 3: parent! child!
Trial 4: child! parent!
Trial 5: parent! child!
Trial 6: child! parent!
Trial 7: chi parent! ld!
Trial 8: parent! child!
Trial 9: child! parent!
```

```
Set parent as winner
Trial 0: child! parent!
Trial 1: parent! child!
Trial 2: parent! child!
Trial 3: parent! child!
Trial 4: child! parent!
Trial 5: parent! child!
Trial 6: parent! child!
Trial 7: child! parent!
Trial 8: parent! child!
Trial 9: parent! child!
```

问题：

在父进程优先的情况下，偶尔会有子进程先于父进程打印到屏幕的情况出现；在子进程优先的

情况下，偶尔也会有父进程先于子进程打印到屏幕的情况。解释可能的原因。

答：

调用了 `yield()` 时，虽然父进程放弃了 CPU，但下一个得到 CPU 的不一定是该子进程，最终还是父进程先继续执行。父进程没调用 `yield()` 时，有可能父进程也要被迫放弃 CPU，随后子进程又在父进程之前被执行。所以偶尔也会看到一个进程输出到一半时，被另一个进程打断了，而出现像 `par child! ent!` 这种情况。

proj2

1.线程支持

多线程的实现分为两部分，一部分为库函数，一部分为系统调用。库函数要调用系统调用，完成一个较为宏观且完整的功能。

1.1 库函数

1.1.1 库函数 xthread_create

```
int xthread_create(int * tid, void * (* start_routine)(void *), void * arg);
```

子线程的用户栈在父进程的堆空间中，该空间由 xthread_create 负责申请，大小为 4k 字节，将该地址传给系统调用 clone，clone 在创建子线程时将该空间作为子线程的用户栈。

在堆中申请的空间不一定页对齐，而为了用户栈地址页对齐。那么申请两倍的空间，使用其中页对齐的部分。

为了记录所申请空间的起始地址，在 PCB 中添加变量 uint tstack，记录子线程用户栈的真实起始地址。

start_routine 函数的返回值保存在 eax 寄存器中。

xthread_create 的任务是将申请到的空间地址传给系统调用 clone，由 clone 完成子线程的创建。

xthread_create 完成代码：

```
int xthread_create(int * tid, void * (* start_routine)(void *), void * arg)
{
    // add your implementation here ...
    void *stack;
    stack = malloc(PGSIZE*2);
    *tid = clone(start_routine, stack, arg);
    if(*tid > -1) return 1;
    return -1;
}
```

1.1.2 库函数 xthread_exit

```
void xthread_exit(void * ret_val_p) ;
```

因线程可以不主动调用 xthread_exit 退出，而其函数通过 return 返回，那么设置线程函数的默认返回地址是 0xffffffff（放入栈中返回地址的位置），这是系统内核区，操作系统会捕获

此错误，我们可以对他进行特殊处理。即在 trap 中捕获它，让线程调用 xthread_exit 进行退出，退出码从 eax 寄存器中取并传给 xthread_exit 函数。

xthread_exit () 完整代码：

```
void xthread_exit(void * ret_val_p)
{
    // add your implementation here ...
    thread_exit(ret_val_p);
}
```

并修改 trap.c 中的 trap 函数

```
.....
//PAGEBREAK: 13
default:
    if(tf->eip==0xffffffff){
        thread_exit((void*)myproc()->tf->eax);
        break;
    }
.....
```

1.1.3 库函数 xthread_join

```
void xthread_join(int tid, void ** retval);
```

调用此 API 的线程会等待，直到 tid 线程顺利结束。tid 线程的返回值被保存在 retval 指向的内存中，并把 join 传回的 tid 线程的用户栈空间释放。

xthread_join 完整代码：

```
void xthread_join(int tid, void ** retval)
{
    // add your implementation here ...
    void *stack;
    join(tid, retval, &stack);
    free(stack);
}
```

1.2 系统调用

1.2.1 系统调用 clone

```
int clone(void * (*fn)(void *), void* stack, void *arg);
```

clone 要在操作系统内核空间中申请一个 PCB，即通过 allocproc() 函数申请一个 PCB。用 PCB 的域 tstack 记录在 thread_create 中申请到的空间的起始地址，将来 join 通过 tstack 域知道以前申请空间的地址，使该空间得以释放。

随后取其中页对齐的 4K 字节作为线程的用户栈。

```
if((uint)stack % PGSIZE)
    stack = stack + (4096 - (uint)stack % PGSIZE);
```

需要注意，申请的空间是用低地址端代表，而用户栈是从高地址向低地址生长。并预留 8 字节，其中 4 字节是子线程最开始运行的函数的参数 arg，另外 4 字节是该函数的返回地址。

```
np->tf->esp = (uint)stack+PGSIZE - 8;
```

接着把参数放入子线程的用户栈预留的空间中。

```
ustack[0] = 0xffffffff; ustack[1] = (uint)arg;
copyout(np->pgdir, np->tf->esp, ustack, 2*4);
```

将子线程的 eip 指向它要执行的第一个函数的位置。并克隆父线程的其他信息。

```
np->tf->eip = (uint)fn;
```

clone() 完整代码如下，（参考 fork 和 exec 函数）：

```
int clone(void * (*fn)(void *), void* stack, void *arg){
    uint ustack[2];
    int i, pid;
    struct proc *np;
    struct proc *curproc = myproc();
    if((np=allocproc())==0){
        return -1;
    }
    np->tstack = (uint)stack;
    np->whoJoinMe = -2;
    if((uint)stack % PGSIZE)
        stack = stack + (4096 - (uint)stack % PGSIZE);
    np->pgdir = curproc->pgdir;
    np->sz = curproc->sz;
    np->parent = curproc->parent;
```

```

*np->tf = *curproc->tf;
np->tf->esp = (uint)stack+PGSIZE - 8;
ustack[0] = 0xffffffff; ustack[1] = (uint)arg;
copyout(np->pgdir, np->tf->esp, ustack, (2)*4);
np->tf->eax = 0;
np->tf->eip = (uint)fn;
np->tf->ebp = np->tf->esp;
for(i = 0; i < NOFILE; i++) {
    if(curproc->ofile[i])
        np->ofile[i] = filedup(curproc->ofile[i]);
}
np->cwd = idup(curproc->cwd);
pid = np->pid;
np->state = RUNNABLE;
safestrcpy(np->name, curproc->name, sizeof(curproc->name));
return pid;
}

```

1.2.2 系统调用 thread_exit

thread_exit 的任务是把线程的返回码放到 eax 寄存器，和先把本线程状态置为 ZOMBIE，然后唤醒正在等着 join 自己的其他线程，并且这样的线程最多只能有一个。因为 join 完之后被等线程的用户栈空间被释放。然后调用 sched 切换到其他线程，且进入 sched 之前必须先落锁。

并调整 exit() 函数，使主线程退出之前，清理其所有子线程。

```

for(p = ptable.proc; p < &ptable.proc[NPROC]; p++){
    if(p->parent == curproc->parent && p!=curproc && p->pgdir == curproc->pgdir){
        kfree(p->kstack);
        p->kstack = 0;
        p->pid = 0;
        p->killed = 0;
        p->parent = 0;
        p->name[0] = 0;
        p->tstack = 0;
        p->state = UNUSED; //放后面
    }
}

```

thread_exit 完整代码如下：

```

void thread_exit(void *ret){
    struct proc * curproc = myproc();
    myproc()->tf->eax = (uint)ret;
}

```

```

    acquire(&ptable.lock);
    curproc->state = ZOMBIE;
    struct proc * p;
    for(p=ptable.proc; p<&ptable.proc[NPROC]; p++){
        if(p->state == SLEEPING && p->pid == curproc->whoJoinMe){
            wakeup1(p);        break;
        }
    }
    sched();
}

```

完整 exit() 如下:

```

void exit(void)
{
    struct proc *curproc = myproc();
    struct proc *p;
    int fd;

    if(curproc == initproc)
        panic("init exiting");

    // Close all open files.
    for(fd = 0; fd < NOFILE; fd++){
        if(curproc->ofile[fd]){
            fileclose(curproc->ofile[fd]);
            curproc->ofile[fd] = 0;
        }
    }

    begin_op();
    iput(curproc->cwd);
    end_op();
    curproc->cwd = 0;

    acquire(&ptable.lock);
    for(p = ptable.proc; p < &ptable.proc[NPROC]; p++){
        if(p->parent == curproc->parent && p!=curproc && p->pgdir == curproc->pgdir){
            kfree(p->kstack);
            p->kstack = 0;
            p->pid = 0;
            p->killed = 0;
            p->parent = 0;

```

```

    p->name[0] = 0;
    p->tstack = 0;
    p->state = UNUSED; //放后面
}
}
// Parent might be sleeping in wait().
wakeup1(curproc->parent);

// Pass abandoned children to init.
for(p = ptable.proc; p < &ptable.proc[NPROC]; p++){
    if(p->parent == curproc){
        p->parent = initproc;
        if(p->state == ZOMBIE)
            wakeup1(initproc);
    }
}

// Jump into the scheduler, never to return.
curproc->state = ZOMBIE;
sched();
panic("zombie exit");
}

```

1.2.3 系统调用 join

进入 join，首先找到被等线程 tid，设置 tid 的 PCB 的 whoJoinMe 为本线程的 pid，然后等待线程 tid 状态变成 ZOMBIE，取出存在 eax 寄存器的返回值，并带回线程 tid 的用户栈空间地址。

采用睡眠-唤醒机制。本线程在第一次扫描中若线程 tid（首先此线程要存在）还不是 ZOMBIE 状态，则主动 sleep，等待线程 tid 顺利退出时唤醒自己。

join() 完整代码（参考 wait()）：

```

void join(int tid, void **ret_p, void **stack){
    struct proc * curproc = myproc();
    struct proc *p;
    int havekids; //, pid;
    acquire(&ptable.lock);
    for(;;){
        // Scan through table looking for zombie children.
        havekids = 0;
        for(p = ptable.proc; p < &ptable.proc[NPROC]; p++){
            if(p->pid == tid){
                p->whoJoinMe = curproc->pid;
            }
        }
    }
}

```

```

    else {
        continue;
    }
    havekids = 1;
    if(p->state == ZOMBIE){
        // Found one.
        *((int*)((int*)stack))=(p->tstack);
        *((int*)((int*)ret_p)) = p->tf->eax;
        kfree(p->kstack);
        p->kstack = 0;
        p->pid = 0;
        p->parent = 0;
        p->name[0] = 0;
        p->killed = 0;
        p->state = UNUSED;
        release(&ptable.lock);
        return;
    }
}
// No point waiting if we don't have any children.
if(!havekids || curproc->killed){
    release(&ptable.lock);
    return ;
}
// Wait for children to exit. (See wakeup1 call in proc_exit.)
sleep(curproc, &ptable.lock); //DOC: wait-sleep
}
}

```

1.3 运行结果

测试用例详参 threadtest.c 文件。

运行结果：

```
dengxiao@VM-199-186-ubuntu: ~/proj2-revise
ptr1 - ptr2 = 16
Return value 123

----- Test Thread Count -----
Child process created 60 threads
Parent process created 61 threads
$
$
$ threadtest

----- Test Return Value -----
Child thread 1: count=3
Child thread 2: count=3
Main thread: thread 1 returned 2
Main thread: thread 2 returned 3

----- Test Stack Space -----
ptr1 - ptr2 = 16
Return value 123

----- Test Thread Count -----
Child process created 60 threads
Parent process created 61 threads
$
$
$ threadtest

----- Test Return Value -----
Child thread 1: count=3
Child thread 2: count=3
Main thread: thread 1 returned 2
Main thread: thread 2 returned 3

----- Test Stack Space -----
ptr1 - ptr2 = 16
Return value 123

----- Test Thread Count -----
Child process created 60 threads
Parent process created 61 threads
$
$
$ threadtest

----- Test Return Value -----
Child thread 1: count=3
Child thread 2: count=3
Main thread: thread 1 returned 2
Main thread: thread 2 returned 3

----- Test Stack Space -----
ptr1 - ptr2 = 16
Return value 123

----- Test Thread Count -----
Child process created 60 threads
Parent process created 61 threads
$ |
```

1.4 问答

问：在你的设计中，struct proc 增加了几个字节？

答：8 字节，分别是 `tstack` 占 4 字节记录子线程用户栈地址，和 `whoJoinMe` 占 4 字节记录哪个线程在等自己结束。

2. 优先级调度

2.1 实验任务

`xv6` 实现了时间片轮转调度算法。请在此基础上实现一个优先级调度算法，具有三个优先级：

- 优先级 3 是最低优先级，2 次之，1 最高
- 每个进程在创建时默认的优先级是 2
- 每次需要调度时，总是优先级最高的进程被调度
- 具有同一个优先级的多个进程采用轮转调度

为了控制本次实验的复杂度，你不需要将进程划分多个队列（当然这样做也可以，只是麻烦了些），新的进程即使具有高优先级也不必考虑立即为它分配 **CPU**，即不必在时间片未到时剥夺其他进程的 **CPU**。（注：你应该需要在 `struct proc` 结构体中添加一个标识优先级的域）

为了能从界面上看到调度结果，你需要做以下工作才能运行测试程序 `schedtest`：

- 在内核中设置一个状态变量 `display_enabled`，如果它的值为 1，表明每次发生调度时均需要输出信息；否则，不输出。`display_enabled` 的初始值为 0。通过系统调用

```
int enable_sched_display(int i);
```

设置状态变量的值。

- 对任何进程，在它放弃 `cpu` 时，如果 `display_enabled` 为 1，则在屏幕上打印进程编号如

```
cprintf("%d - ",proc->pid)
```

（回想：进程在什么函数中切换回 `scheduler`？）

- 提供一个系统调用可以设置进程的优先级

```
int set_priority(int pid, int prior)
```

其中 `pid` 指进程编号，`prior` 是需要设置的优先级。

2.2 实现方法

1. 在 `proc.c` 中设置全局变量 `int display_enable`，控制每次调度时是否输出相应信息，为 1 输出否则不输出。
2. 在 PCB 中添加进程优先级标记 `int priority`。
3. 添加系统调用 `void enable_sched_display(int i)`用以调整 `display_enable`。

```
void enable_sched_display(int i){
    display_enable = i;
}
```

4. 添加系统调用 `int set_priority(int pid, int prior)`用以将进程 `pid` 的优先级设置为 `prior`。

```
int set_priority(int pid, int prior){
    struct proc * p;
    acquire(&tickslock);
    for(p=ptable.proc; p<&ptable.proc[NPROC]; p++){
        if(p->pid==pid){
            p->priority = prior;
            release(&tickslock);
            return 0;
        }
    }
    release(&tickslock);
    return 0;
}
```

5. 在 `allocproc` 中，将每个新进程的优先级都默认设置为 2。

```
.....
found:
    p->priority = 2;
    p->state = EMBRYO;
    p->pid = nextpid++;
.....
```

6. 改造调度函数 `scheduler()`。改造其中的寻找部分即可。

从别的进程切到 `scheduler` 时，都是从 `swtch(&(c->scheduler), p->context);`

的下一条语句开始执行，那么使用变量 `nowprior`，表示本次遍历要找 `priority` 等于 `nowprior` 的进程。又因为是带优先级的轮转，而需使用变量 `struct proc *prep` 记录前一次得到执行的进程。每次切回 `scheduler` 时，都将 `nowprior` 设置为 1，并从 `prep` 后的第一个进程开始遍历，即从 `prep + 1` 至 `prep`（包括 `prep`）环状遍历。第一次遍历时如果没找到状态是 `RUNNABLE` 的进程，则调整 `nowprior += 1`，表示下一轮遍历找下一优先级的进程。周而复始。一找到状态为 `RUNNABLE` 且优先级符合本次查找的优先级则切换过去运行该进程。

scheduler() 完整代码:

```
int nowprior = 1;
void scheduler(void) {
    struct proc *p, *prep;
    struct cpu *c = mycpu();
    c->proc = 0;
    prep = &ptable.proc[NPROC] - 1; //第一次时假设是最后一个刚被执行
    p = ptable.proc;
    for(;;){
        sti();
        acquire(&ptable.lock);
        for(;;){ //change nowprior
            for(;; ){
                if(p==prep){ //找了一周了，要么找到可执行进程，要么调整 nowprior 继续找
                    if(p->state == RUNNABLE && p->priority == nowprior){
                        goto found;
                    }
                    else {
                        p = prep + 1;
                        if(! (p < &ptable.proc[NPROC])) p = ptable.proc;
                        break;
                    }
                }
            }
            if(p->state != RUNNABLE || p->priority!=nowprior){
                p++; if(! (p < &ptable.proc[NPROC])) p = ptable.proc;
                continue;
            }
            found:
            c->proc = p;
            switchvm(p);
            p->state = RUNNING;
            prep = p;

            swtch(&(c->scheduler), p->context);
            nowprior = 1;
            p = prep + 1; if(! (p < &ptable.proc[NPROC])) p = ptable.proc;
            //接下来，从 prep+1 循环到 prep
            switchkvm();
            c->proc = 0;
        }
        nowprior++;
        if(nowprior==4){
```

```

        nowprior = 1;
        break;
    }
}
release(&ptable.lock);
}
}

```

7. 为在任何进程放弃 cpu 时打印信息，进程放弃 cpu 时，都会通过 sched() 函数转到 scheduler()，在 `swtch(&p->context, mycpu()->scheduler);` 语句之前处理信息输出，如下。

```

void sched(void)
{
    int intena;
    struct proc *p = myproc();
    if(!holding(&ptable.lock))
        panic("sched ptable.lock");
    if(mycpu()->ncli != 1)
        panic("sched locks");
    if(p->state == RUNNING)
        panic("sched running");
    if(readeflags() & FL_IF)
        panic("sched interruptible");
    intena = mycpu()->intena;
    if(display_enable)
        cprintf("%d - ", myproc()->pid);
    swtch(&p->context, mycpu()->scheduler);
    mycpu()->intena = intena;
}

```

2.3 运行结果

测试用例源码参见 schedtest. 文件，此处略。

```
dengxiao@VM-199-186-ubuntu: ~/proj2-revise
xv6...
cpu0: starting 0
sb: size 1000 nblocks 941 ninodes 200 nlog 30 logstart 2 inodestart 32 bmap start 58
init: starting sh
$ schedtest
=====
Parent (pid=3, prior=1)
Child (pid=4, prior=1) created!
Child (pid=5, prior=2) created!
Child (pid=6, prior=3) created!
Child (pid=7, prior=1) created!
Child (pid=8, prior=2) created!
Child (pid=9, prior=3) created!
Child (pid=10, prior=2) created!
=====
3 - 4 - 7 - 4 - 7 - 4 - 7 - 4 - 7 - 4 - 7 - 4 - 7 - 3 - 4 - 3 - 3 - 5 - 8 - 10 - 5 - 8 - 10 -
- 5 - 8 - 10 - 5 - 8 - 10 - 5 - 8 - 10 - 5 - 8 - 10 - 5 - 3 - 8 - 10 - 3 - 8 - 8 - 3 - 6 - 9 - 6 - 9 -
- 6 - 9 - 6 - 9 - 6 - 9 - 6 - 9 - 6 - 9 - 3 - 6 - 3 -
$ schedtest
=====
Parent (pid=11, prior=1)
Child (pid=12, prior=1) created!
Child (pid=13, prior=2) created!
Child (pid=14, prior=3) created!
Child (pid=15, prior=1) created!
Child (pid=16, prior=2) created!
Child (pid=17, prior=3) created!
Child (pid=18, prior=2) created!
=====
11 - 12 - 11 - 12 - 15 - 12 - 15 - 12 - 15 - 12 - 15 - 12 - 15 - 11 - 15 - 15 - 15 - 15 - 11 - 13 -
16 - 18 - 13 - 16 - 18 - 13 - 16 - 18 - 13 - 16 - 18 - 13 - 16 - 18 - 13 - 16 - 18 - 13 - 16 - 11 -
13 - 11 - 18 - 18 - 11 - 11 - 14 - 17 - 14 - 17 - 14 - 17 - 14 - 17 - 14 - 17 - 14 - 17 - 14 - 11 -
17 -
$ |
```

2.4 问答

问：父进程的优先级为 1，为何有时优先级低的子进程会先于它执行？

答：高优先级进程在等待某触发条件，如 IO 结束，子进程结束等。只能先调度其他包括更低优先级的进程。

问：父进程似乎周期性出现在打印列表中，为什么？

答：因为父进程在 wait 子进程，每隔段时间才有一个子进程执行结束。

问：set_priority 系统调用会否和 scheduler 函数发生竞争条件（race condition）？你是如何解决的？

答：会发生竞争条件，如 set_priority() 执行到一半时，被切到 scheduler，此时无法确定进程优先的修改是否已经完成，所以进入 set_priority() 时先落锁 acquire(&tickslock)，修改完成退出时放锁 release(&tickslock)。

3 综合执行结果

```
dengxiao@VM-199-186-ubuntu: ~/proj2-revise
349+1 records in
349+1 records out
179136 bytes (179 kB, 175 KiB) copied, 0.000844675 s, 212 MB/s
qemu-system-i386 -nographic -drive file=fs.img,index=1,media=disk,format=raw -drive file=xv6.img,index=0,media=disk,format=raw -smp 1 -m 256
xv6...
cpu0: starting 0
sb: size 1000 nblocks 941 ninodes 200 nlog 30 logstart 2 inodestart 32 bmap start 58
init: starting sh
$ schedtest
=====
Parent (pid=3, prior=1)
Child (pid=4, prior=1) created!
Child (pid=5, prior=2) created!
Child (pid=6, prior=3) created!
Child (pid=7, prior=1) created!
Child (pid=8, prior=2) created!
Child (pid=9, prior=3) created!
Child (pid=10, prior=2) created!
=====
3 - 4 - 7 - 4 - 7 - 4 - 7 - 4 - 7 - 4 - 7 - 3 - 4 - 3 - 5 - 8 - 10 - 5 - 8 - 10 - 5 - 8 - 10
- 5 - 8 - 10 - 5 - 8 - 10 - 5 - 8 - 3 - 3 - 5 - 10 - 3 - 5 - 3 - 3 - 6 - 9 - 6 - 9 - 6 - 9
- 6 - 9 - 6 - 3 - 9 - 9 -
$ schedtest
=====
Parent (pid=11, prior=1)
Child (pid=12, prior=1) created!
Child (pid=13, prior=2) created!
Child (pid=14, prior=3) created!
Child (pid=15, prior=1) created!
Child (pid=16, prior=2) created!
Child (pid=17, prior=3) created!
Child (pid=18, prior=2) created!
=====
11 - 12 - 15 - 12 - 15 - 12 - 15 - 12 - 15 - 12 - 15 - 12 - 15 - 11 - 12 - 12 - 11 - 13 - 16 - 18 -
13 - 16 - 18 - 13 - 16 - 18 - 13 - 16 - 18 - 13 - 16 - 18 - 13 - 16 - 11 - 13 - 18 - 11 - 13 - 11 -
14 - 17 - 14 - 17 - 14 - 17 - 14 - 17 - 14 - 17 - 14 - 17 - 11 - 14 - 14 -
$ threadtest

----- Test Return Value -----
Child thread 1: count=3
Child thread 2: count=3
Main thread: thread 1 returned 2
Main thread: thread 2 returned 3

----- Test Stack Space -----
ptr1 - ptr2 = 16
Return value 123

----- Test Thread Count -----
Child process created 60 threads
Parent process created 61 threads
$
$
$
$ threadtest

----- Test Return Value -----
Child thread 1: count=3
Child thread 2: count=3
Main thread: thread 1 returned 2
Main thread: thread 2 returned 3

----- Test Stack Space -----
ptr1 - ptr2 = 16
Return value 123

----- Test Thread Count -----
Child process created 60 threads
Parent process created 61 threads
$
```

[参考资料]

- [1] XV6 文档 <https://th0ar.gitbooks.io/xv6-chinese/content/content/preface.html>
- [2] XV6 源代码阅读-进程线程 https://blog.csdn.net/qg_36347375/article/details/91354349
- [3] unix 编程以及 xv6 系统浅谈（七）线程 <https://juejin.im/post/5dc42dad6fb9a04a680f547d>
- [4] xv6 源码分析（六）：进程调度 https://blog.csdn.net/qg_25426415/article/details/54693730

proj3

1. 信号量

1.1 任务说明

借助 `spinlock`，为 `xv6` 添加信号量的支持。

定义一个信号量结构体 `struct semaphore`，成员自定，然后在内核里开辟一个包含 100 个信号量的空间，假如是 `s[100]`，提供以下**系统调用**给用户程序：

```
int alloc_sem (int v);
```

创建一个初始值为 `v` 的信号量，返回信号量的下标。若返回值为 `-1`，表明分配失败。分配失败的原因可能是初始值为负数或者系统中信号量资源不足。

```
int wait_sem(int i);
```

对信号量 `s[i]` 执行 `wait` 操作；成功返回 `1`，出错返回 `-1`。出错的原因可能是该信号量不存在，如 `i` 不在 `0~99` 之间，或者信号量尚未分配。

```
int signal_sem(int i);
```

对信号量 `s[i]` 执行 `signal` 操作；成功返回 `1`，出错返回 `-1`。出错的原因同上。

```
int dealloc_sem(int i);
```

删除信号量 `s[i]`，将 `s[i]` 标记为未分配。同时将所有等待 `s[i]` 的进程终止（终止进程时参考 `kill` 的实现）。成功返回 `1`，失败返回 `-1`。失败的情况包括下标不在合法范围内以及信号量未被分配。

添加 `semtest.c` 为用户命令，运行 `semtest` 命令。

- 第 1 部分你应当看到两个 `success`。
- 第 2 部分应当有以下规律：当信号量值为 `1` 时，只有一个进程能输出，所以输出不会杂乱；当信号量值为 `2` 时，有两个进程可以同时输出，所以输出会交叉；以此类推。
- 第 3 部分，你应当能够看到一行字 “parent leaving”。

请在实验报告中回答：

关于你实现的 `wait_sem`，如果多个进程同时执行 `wait_sem`，但 `wait` 的具体信号量不同，它们能并发执行吗？（你应该用一个 `spinlock` 去保护一个信号量，另外用一个 `spinlock` 保护资源分配）

1.2 解决方案

/*以下实现的不是正宗的信号量，因为 wait 是忙等，而没有采用阻塞，这是后来考试时才意识到*/

1) 定义信号量结构体 struct semaphore

在 spinlock.h 中添加 struct semaphore 的定义

```
// in spinlock.h
struct semaphore{
    int used;           //此信号量是否已被使用，1 已被使用，否则没被使用
    int semap;          //信号量的值
    int whichPidAlloc;  //已被使用时，记录是哪个进程创建此信号量，以在该进程销毁时，
                        //释放此信号量
    struct spinlock lock; //要修改此信号量要持有它对应的锁
};
```

2) 在内核里开辟一个包含 100 个信号量的空间 s[100]

在 main.c 中创建 100 个信号量 s[100]，并在 mpmain（）中（开机时执行）初始化这些信号量为未分配的，并初始化其对应的 spinlock。

```
// in main.c
struct semaphore s[100];
static void mpmain(void)
{
    cprintf("cpu%d: starting %d\n", cpuid(), cpuid());
    /* -----初始化新添加的 semaphore-----*/
    int i;
    for(i=0; i<100; i++){
        s[i].used = 0;
        initlock(&s[i].lock, "anyway");
    }/*----初始化完毕-----*/
    idtinit();           // load idt register
    xchg(&(mycpu()->started), 1); // tell startothers() we're up
    scheduler();         // start running processes
}
```

3) 提供 alloc_sem(int v) 系统调用

功能：alloc_sem 创建一个初始值为 v 的信号量，返回信号量的下标。若返回值为-1，表明分

配失败。分配失败的原因可能是初始值为负数或者系统中信号量资源不足。

从序号 0 开始遍历所有信号量，找到空闲的信号量就马上分配出去。访问每个信号量前要拿到该信号量对应的 spinlock（以下简称锁），以防止一个进程在找到空闲信号量 i 且在分配出去的过程中（正在将其 `used` 置 1），另一个进程也将信号量 i 分配出去。即保证了机器上的某一时刻，至多只有一个进程在修改某个信号量的状态，但可以并行访问不同的信号量。

后到的进程只能跟在先到进程的后面遍历，但先到的不一定先得，因为可能先到的进程检查过某个信号量状态之后，其他进程才释放该信号量，而正好被后来的进程碰到了。

```
// in proc.c
int alloc_sem(int v){
    if (v < 0)
        return -1;
    int i;
    //cprintf("alloc %d and it's used is %d\n", 1, s[1].used);
    for (i = 0; i < 100; i++) {
        acquire(&s[i].lock);
        if (s[i].used == 0) {
            s[i].used = 1;
            s[i].whichPidAlloc = myproc()->pid;
            s[i].semap = v;
            release(&s[i].lock);
            return i;
        }
        release(&s[i].lock);
    }
    return -1;
}
```

4) 提供 `wait_sem(int i)` 系统调用

功能：对信号量 `s[i]` 执行 `wait` 操作；成功返回 1，出错返回 -1。出错的原因可能是该信号量不存在，如 i 不在 $0 \sim 99$ 之间，或者信号量尚未分配。

在 PCB 中添加变量 `waitSignal`，表示本进程正在等哪个信号量，以在释放该信号量时杀死此进程（遍历进程列表找到此进程）。在 `allocproc(void)` 中将其初始化为 -1，表示本进程现在没在等任何信号量。

```
// in proc.h
struct proc {
    .....
```

```

int waitSignal;           //waiting which signal
.....
};

```

进入 wait_sem 先拿信号量 i 的锁（以下有时称为 s[i] 的锁），将本进程的 waitSignal 置为该信号量的序号（信号量 id），然后放锁，原因见“6) dealloc_sem(int i) 系统调用”。

然后再拿 s[i] 的锁后，判断信号量是不是已经被其他进程释放了做相应处理。然后读信号量 i 的值，若大于 0 则将其减一后，结束本次等待，放锁离开；否则放锁后继续等待。先拿信号量的锁在执行有效操作避免了多个进程同时修改该信号量的状态。

这里每个信号量都有其对应的锁，从而让机器能同时修改不同信号量的状态。

```

// in proc.c
int wait_sem(int i){
    if ((i < 0 || i > 99)) return -1;
    acquire(&s[i].lock);
    myproc()->waitSignal = i;
    release(&s[i].lock);
    //cprintf("add %d to wait signal %d queue\n", myproc()->pid, i);
    int a;
restart:
    acquire(&s[i].lock);
    if(s[i].used == 0){
        release(&s[i].lock);
        return -1;
    }
    a = s[i].semaph;
    if (a <= 0){
        release(&s[i].lock);
        goto restart;
    }
    s[i].semaph--;
    myproc()->waitSignal = -1;
    release(&s[i].lock);
    return 1;
}

```

5) 提供 signal_sem(int i) 系统调用

功能：对信号量 s[i] 执行 signal 操作，成功返回 1，出错返回-1。

先拿 s[i] 的锁，判断 s[i] 是否存在，然后执行 s[i].semaph++。拿 s[i] 的锁防止同时有多个

进程修改 `s[i]` 的状态，而导致出错。

```
// in proc.c
int signal_sem(int i){
    if (i < 0 || i > 99) {
        return -1;
    }
    acquire(&s[i].lock);
    if(s[i].used == 0){
        release(&s[i].lock);
        return -1;
    }
    s[i].semap++;
    release(&s[i].lock);
    //cprintf("signal sema %d, and it's value is %d\n", i, s[i].semap);
    return 1;
}
```

6) 提供 `dealloc_sem(int i)` 系统调用

功能：删除信号量 `s[i]`，将 `s[i]` 标记为未分配。同时将所有等待 `s[i]` 的进程终止。成功返回 1，失败返回 -1。失败的情况包括下标不在合法范围内以及信号量未被分配。

遍历找哪个进程在等待 `s[i]` 前要拿 `s[i]` 的锁，并且其他进程（假设该进程是 A）进入 `wait_sem` 也要拿 `s[i]` 的锁之后才将它的 `waitSignal`（正在等哪个信号量）标为“i”。这里的分析假设进程 A 要等的信号量就是此时 `dealloc_sem` 要释放的信号量 i，否则不会出现竞争条件。

两边都要先拿 `s[i]` 的锁，目的是防止 `dealloc_sem` 在遍历进程列表时，访问进程 A 之后，进程 A 才标记它正在等 `s[i]`，而导致在 `dealloc_sem` 中没有把进程 A 杀死，最终可能发生 `s[i]` 已被释放而进程 A 还在等 `s[i]`（这也可以通过在 `wait_sem` 中每次访问信号量时判断该信号量是否已被释放来解决）。

简而言之，两边都要拿锁，保证了要么 A 先把自己标记为正在等 `s[i]`，然后被 `dealloc_sem` 杀死；要么 `dealloc_sem` 先把 `s[i]` 释放，然后进程 A 发现 `s[i]` 不存在，而自己结束等待。

遍历进程列表，将所有正在等 `s[i]` 的进程杀死，然后把 `s[i]` 标为未使用。这里杀死进程后要把它子孙过继给 `initproc`，参考 `kill()` 和 `exit()`。

可以注意到这里有一个模棱两可的问题，万一 `s[i]` 被进程 B 释放后马上又被进程 C 给占用了，本来进程 A 在等的 `s[i]` 是和进程 B 分享的那个 `s[i]`，而不是现在这个已经被 B 释放后被 C 占用的 `s[i]`，这可以算一种出错。但要是退一步说，反正有个 `s[i]` 让进程 A 等就行了，不用管 A 开始等的那一刻 `s[i]` 是被谁占用的。我这里没有进一步解决这个问题，因为它或许就不是一个问题。

```
// in proc.c
int dealloc_sem(int i){
```

```

struct proc *p, *p2;
if (i < 0 || i > 99) {
    return -1;
}
acquire(&s[i].lock);
if(s[i].used == 0){
    release(&s[i].lock);
    return -1;
}
for (p = ptable.proc; p < &ptable.proc[NPROC]; p++) {
    if (p->waitSignal == i && p->pid != myproc()->pid && p->state != UNUSED){
        p->state = ZOMBIE;
        for (p2 = ptable.proc; p2 < &ptable.proc[NPROC]; p2++) {
            if (p2->parent == p){
                p2->parent = initproc;
                if (p2->state == ZOMBIE)
                    wakeup1(initproc);
            }
        }
    }
}
s[i].used = 0;
release(&s[i].lock);
return 1;
}

```

1.3 运行结果

测试代码参见 semtest.c 文件。

— □ ×

^

1.4 问答

提问：

关于你实现的 wait_sem，如果多个进程同时执行 wait_sem，但 wait 的具体信号量不同，它们能并发执行吗？

回答：可以并行执行，因为每个信号量有其特定的 spinlock，要修改信号量 i 的状态只需持有信号量 i 对应的锁。

2. 消息传递

2.1 任务说明

xv6 没有实现消息传递机制，你需要添加这个支持，实现以下系统调用。

```
int msg_send (int pid, int a, int b, int c);
```

将 a, b, c 三个整数发送给编号为 pid 的进程，直到接收者进程执行完 receive 才返回。若接收者尚未执行 receive，则进入阻塞状态。成功返回 1，失败返回-1。失败的情况包括系统资源不足或者不存在编号为 pid 的进程。

```
int msg_receive(int *a, int *b, int *c);
```

接收一个消息(三个整数)，返回发送者的进程编号。如果未接收到任何消息，则陷入阻塞状态。

你可以用前面实现的信号量，但请注意：

- 信号量应当用时分配，不可以提前分配
- 当进程消亡时，它所占用的信号量资源应当释放
- 先执行 receive，则 receive 会阻塞，直到有进程 send
- 先执行 send，则 send 会阻塞，直到有进程 receive

测试程序为 msgtest.c，请添加其为用户命令。

- 第 1 部分测试是否在进程结束时释放信号量资源，你应当能够看到 120 组打印；
 - 第 2 部分在此基础上测试多个子进程同时向父进程发消息是否出错，应当能看到有 20 个子进程编号；
 - 第 3 部分测试较长字符串的传递，应该看到 Hello World! Good Morning! 字符串。
- 请在实验报告中回答：

如果 receive 先执行，但 send 未执行，你是如何让 receive 阻塞的（或者说阻塞到哪个信号量上了）？如果 send 先执行，但 receive 未执行，你是如何让 send 阻塞的？

2.2 解决方案

1) 消息传递中转空间

方案 1（实现）：

在 PCB 中用 int destProc 表示目标进程的 pid。并在内核开辟一块用于消息传递中转的空间 int message[NPROC][3]，每块空间对应一个发送进程（其实把 message 的每一块分别放到各个进程的 PCB 中，所占的空间大小是一样）。

```
// in proc.h
```

```
struct proc {
    .....
    int destProc;           //destinational process of message
};
```

```
// in proc.c
int message[NPROC][3];
```

方案 2（未实现）：

只开一块中转空间，内容包括消息内容用 a[3] 表示，发送进程的 pid 用 sourceProc 表示，目标进程的 pid 用 destProc 表示，并用一个 spinlock 来保证至多只有一对进程在使用此中间空间进行消息传递。这是最节省中转空间的方案，但是同时最多只能有一对进程能进行消息传递，效率不高。

方案 1 的实现使用 ptable.lock 来保证至多只有一个进程在执行 msg_send 或 msg_receive，但可以多个进程同时处于阻塞状态，即可以有多个进程同时处于发送或接收的等待状态。

destProc 在 allocproc() 中初始化为-1，表示没在进行消息传递。

2) 系统调用 msg_send(int pid, int a, int b, int c)

功能：将 a, b, c 三个整数发送给编号为 pid 的进程，直到接收者进程执行完 receive 才返回。若接收者尚未执行 receive，则进入阻塞状态。成功返回 1，失败返回-1。

进入 msg_send 后先获得 ptable.lock，然后把消息 a, b, c 依次放入本发送进程对应的 message 空间中，并将 destProc 置为目标进程 pid。

随后遍历进程列表，找到目标进程将其唤醒（只有在其处于 SLEEPING 状态时，才会被真正唤醒）。若目标进程不存在，则置本进程的 destProc 为-1，结束消息传递，放锁离开；若目标进程存在，则发送进程无条件进入 SLEEPING 状态，等待目标进程接收消息完毕后将发送进程唤醒，随后发送进程放锁离开。

因为接收进程进入 msg_receive 之后也要先拿锁才能接收消息，接收进程只有在发送进程进入 SLEEPING 状态之后才有可能拿到锁，所以接收进程不会在发送进程进入 SLEEPING 状态之前接收完毕。

```
// in proc.c
int msg_send(int pid, int a, int b, int c){
    //cprintf("I am %d come to send message to %d\n", myproc()->pid, pid);
    struct proc *curproc;
    struct proc *p;
    int flag = 0;
    curproc = myproc();
    acquire(&ptable.lock);
```

```

//cprintf("I am %d get lock successfully in send\n", curproc->pid);
int k = (curproc - ptable.proc); //该距离以间隔的单元格数为单位，而不是以字节为单位
message[k][0] = a; message[k][1] = b; message[k][2] = c;
curproc->destProc = pid;
for(p=ptable.proc; p<&ptable.proc[NPROC]; p++){
    if(p->pid == pid && p->state!=UNUSED){
        wakeup1(p);
        flag = 1;
        break;
    }
}
if(flag == 0){
    curproc->destProc = -1;
    release(&ptable.lock);
    return -1;
} // dest process doesn't exist
sleep(curproc, &ptable.lock);
release(&ptable.lock);
return 1;
}

```

3) 系统调用 msg_receive(int *a, int *b, int *c)

功能：接收进程调用 msg_receive 接收一个消息(三个整数)，返回发送者的进程编号。如果未接收到任何消息，则陷入阻塞状态。

接收进程进入 msg_receive 后要先拿锁，随后遍历进程列表，查看它们的 PCB 里的 destProc，以判断它们哪个要给自己发消息。

若找到发送进程，则马上去接收，接收完毕后将发送进程 PCB 的 destProc 置-1，表示本次消息传递结束。随后放锁，将发送进程唤醒（此时发送进程必定处于 SLEEPING 状态，因为只有发送进程处于 SLEEPING 接收进程才能拿到锁）后离开。

若没找到发送进程，说明发送进程还没进行消息发送，那么接收进程进入 SLEEPING 状态，等待发送进程执行 msg_send 时将接收进程唤醒（发送进程将消息放入指定空间后都会无条件将接收进程唤醒）。要么接收进程没执行 msg_receive，要么接收进程已经处于 SLEEPING，发送进程的 msg_send 才能拿到锁。所以对于接收进程来说，如果发送进程已经发送消息，那直接接收消息即可；如果发送进程还没发送消息，那等待发送进程唤醒然后再去接收消息即可。

```

// in proc.c
int msg_receive(int *a, int *b, int *c){
    //cprintf("I am %d come to receive message\n", myproc()->pid);
    struct proc *p, *curproc = myproc();

```

```

int mypid = myproc()->pid, k;
acquire(&ptable.lock);
for(p=ptable.proc; p<&ptable.proc[NPROC]; p++){
    if(p->destProc == mypid){
        goto recei;
    }
}
sleep(curproc, &ptable.lock);
for(p=ptable.proc; p<&ptable.proc[NPROC]; p++){
    if(p->destProc == mypid){
        goto recei;
    }
}
recei:
k = (p - ptable.proc); //该距离以间隔的单元格数为单位，而不是以字节为单位
*a = message[k][0]; *b = message[k][1]; *c = message[k][2];
p->destProc = -1;
release(&ptable.lock);
wakeup1(p);
return p->pid;
}

```

4) 修改 exit(), 在进程消亡时释放其申请的信号量

遍历所有信号量，查看哪个是由此消亡进程申请的，将其释放。

```

// in proc.c
void exit(void)
{
    struct proc *curproc = myproc();
    struct proc *p;
    int fd;
    //cprintf("I am %d in exit\n", myproc()->pid);
    if (curproc == initproc)
        panic("init exiting");

    // Close all open files.
    for (fd = 0; fd < NOFILE; fd++)
    {
        if (curproc->ofile[fd])
        {
            fileclose(curproc->ofile[fd]);
        }
    }
}

```

```

    curproc->ofile[fd] = 0;
}
}

begin_op();
iput(curproc->cwd);
end_op();
curproc->cwd = 0;

acquire(&ptable.lock);
/*----- delete signal which was alloced by me -----*/
int kk;
for(kk=0; kk<100; kk++){
    if(s[kk].whichPidAlloc == myproc()->pid && s[kk].used == 1){
        s[kk].used = 0;
        initlock(&s[kk].lock, "anyway");
    }
}
/*-----finish delete signal-----*/
// Parent might be sleeping in wait().
wakeup1(curproc->parent);

// Pass abandoned children to init.
for (p = ptable.proc; p < &ptable.proc[NPROC]; p++)
{
    if (p->parent == curproc)
    {
        p->parent = initproc;
        if (p->state == ZOMBIE)
            wakeup1(initproc);
    }
}

// Jump into the scheduler, never to return.
curproc->state = ZOMBIE;
sched();
panic("zombie exit");
}

```

2.3 运行结果

测试代码详见 msgtest.c 文件。

```
dengxiao@VM-199-186-ubuntu: ~/proj3-revise
dd if=bootblock of=xv6.img conv=notrunc
1+0 records in
1+0 records out
512 bytes copied, 0.000143106 s, 3.6 MB/s
dd if=kernel of=xv6.img seek=1 conv=notrunc
351+1 records in
351+1 records out
180068 bytes (180 kB, 176 KiB) copied, 0.000792174 s, 227 MB/s
qemu-system-i386 -nographic -drive file=fs.img,index=1,media=disk,format=raw -drive file=xv6.img,index=0,media=disk,format=raw -smp 1 -m 256
xv6...
cpu0: starting 0
sb: size 1000 nblocks 941 ninodes 200 nlog 30 logstart 2 inodestart 32 bmap start 58
init: starting sh
$ msgtest

child(sender,child+3): 23(3,26) 4(3,7) 5(3,8) 6(3,9) 7(3,10) 8(3,11) 9(3,12) 10(3,13) 11(3,14) 12(3,15) 13(3,16) 14(3,17) 15(3,18) 16(3,19) 17(3,20) 18(3,21) 19(3,22) 20(3,23) 21(3,24) 22(3,25) 43(3,46) 24(3,27) 25(3,28) 26(3,29) 27(3,30) 28(3,31) 29(3,32) 30(3,33) 31(3,34) 32(3,35) 33(3,36) 34(3,37) 35(3,38) 36(3,39) 37(3,40) 38(3,41) 39(3,42) 40(3,43) 41(3,44) 42(3,45) 63(3,66) 44(3,47) 45(3,48) 46(3,49) 47(3,50) 48(3,51) 49(3,52) 50(3,53) 51(3,54) 52(3,55) 53(3,56) 54(3,57) 55(3,58) 56(3,59) 57(3,60) 58(3,61) 59(3,62) 60(3,63) 61(3,64) 62(3,65) 83(3,86) 64(3,67) 65(3,68) 66(3,69) 67(3,70) 68(3,71) 69(3,72) 70(3,73) 71(3,74) 72(3,75) 73(3,76) 74(3,77) 75(3,78) 76(3,79) 77(3,80) 78(3,81) 79(3,82) 80(3,83) 81(3,84) 82(3,85) 103(3,106) 84(3,87) 85(3,88) 86(3,89) 87(3,90) 88(3,91) 89(3,92) 90(3,93) 91(3,94) 92(3,95) 93(3,96) 94(3,97) 95(3,98) 96(3,99) 97(3,100) 98(3,101) 99(3,102) 100(3,103) 101(3,104) 102(3,105) 123(3,126) 104(3,107) 105(3,108) 106(3,109) 107(3,110) 108(3,111) 109(3,112) 110(3,113) 111(3,114) 112(3,115) 113(3,116) 114(3,117) 115(3,118) 116(3,119) 117(3,120) 118(3,121) 119(3,122) 120(3,123) 121(3,124) 122(3,125)
messages received from :124 125 126 127 128 129 130 131 132 133 134 135 136 137 138 139 140 141 142 143
Hello World! Good Morning!$ msgtest

child(sender,child+3): 1146(145,149) 65(145,168) 147(145,150) 148(145,151) 149(145,152) 150(145,153) 151(145,154) 152(145,155) 153(145,156) 154(145,157) 155(145,158) 156(145,159) 157(145,160) 158(145,161) 159(145,162) 160(145,163) 161(145,164) 162(145,165) 163(145,166) 164(145,167) 1166(145,169) 85(145,170) 167(145,171) 14168(145,171) 5,188) 169(145,172) 170(145,173) 171(145,174) 172(145,175) 173(145,176) 174(145,177) 175(145,178) 176(145,179) 177(145,180) 178(145,181) 179(145,182) 180(145,183) 181(145,184) 182(145,185) 183(145,186) 184(145,187) 2186(145,189) 05(145,208) 187(145,190) 188(145,191) 189(145,192) 190(145,193) 191(145,194) 192(145,195) 193(145,196) 194(145,197) 195(145,198) 196(145,199) 197(145,200) 198(145,201) 199(145,202) 200(145,203) 201(145,204) 202(145,205) 203(145,206) 204(145,207) 2206(145,209) 25(207(145,210) 145,228) 208(145,211) 209(145,212) 210(145,213) 211(145,214) 212(145,215) 213(145,216) 214(145,217) 215(145,218) 216(145,219) 217(145,220) 218(145,221) 219(145,222) 220(145,223) 221(145,224) 222(145,225) 223(145,226) 224(145,227) 226(145,229) 245(1227(145,230) 45,248) 228(145,231) 229(145,232) 230(145,233) 231(145,234) 232(145,235) 233(145,236) 234(145,237) 235(145,238) 236(145,239) 237(145,240) 238(145,241) 239(145,242) 240(145,243) 241(145,244) 242(145,245) 243(145,246) 244(145,247) 265(145,268) 246(145,249) 247(145,250) 248(145,251) 249(145,252) 250(145,253) 251(145,254) 252(145,255) 253(145,256) 254(145,257) 255(145,258) 256(145,259) 257(145,260) 258(145,261) 259(145,262) 260(145,263) 261(145,264) 262(145,265) 263(145,266) 264(145,267)
messages received from :267 268 269 270 271 272 273 274 275 276 277 278 279 280 281 282 283 284 285 266
Hello World! Good Morning!$ |
```

2.4 问答

提问：

如果 receive 先执行，但 send 未执行，你是如何让 receive 阻塞的（或者说阻塞到哪个信号量上了）？

回答：

阻塞在 ptable.lock 上，等待发送进程在 msg_send 里把自己唤醒。因为进入 msg_send 和 msg_receive 后都要先拿到 ptable.lock，所以它们不能同时执行。msg_send 后面执行时一定能把接收进程唤醒来接收消息。

如果 send 先执行，但 receive 未执行，你是如何让 send 阻塞的？

回答：

在 msg_send 中，不管 receive 是否已经执行，都要阻塞在 ptable.lock 上，等待接收进程接收完毕后把发送进程唤醒，同理，接收进程一定能将发送进程唤醒。

3. 综合运行结果

[illegible]

```

$ semtest
illegal input test: success!
alloc-dealloc test: success!
created semaphores with indices 0 1 2
testing semaphore 0 with initial value 1
-24--24--24--24--24--24--24--24--24--24--24--24--24--24--24--24--
leaving(24)
-25--25--25--25--25--25--25--25--25--25--25--25--25--25--25--
leaving(25)
-26--26--26--26--26--26--26--26--26--26--26--26--26--26--26--
leaving(26)
-27--27--27--27--27--27--27--27--27--27--27--27--27--27--27--
leaving(27)
-28--28--28--28--28--28--28--28--28--28--28--28--28--28--28--
leaving(28)
testing semaphore 1 with initial value 2
-29--30--29--30--29--30--29--30--29--30--29--30--29--30--29--30--29--30--29--30--29--
leaving(29)

leavin-31-g(30)
-32--31--32--31--32--31--32--31--32--31--32--31--32--31--32--31--32--31--32--31--32--
-31--32--31--32--31--32--31--32--31--32--31--32--31--32--31--32--31--32--31--32--
leaving(31)
-33-
leaving(32)
-33--33--33--33--33--33--33--33--33--33--33--33--33--33--33--33--
leaving(33)
testing semaphore 2 with initial value 3
-34--35--36--34--35--36--34--35--36--34--35--36--34--35--36--34--35--36--34--35--36--34--
-35--36--34--35--36--34--35--36--34--35--36--34--35--36--34--35--36--34--35--36--34--35--
-36--34--35--36--34--35--36--34--35--36--
leaving(34)

lea
leaving(36)
-37--38--ving(35)
-37--38--37--38--37--38--37--38--37--38--37--38--37--38--37--38--37--38--37--38--37--
-38--37--38--37--38--37--38--37--38--37--38--37--38--37--38--
leaving(37)

leaving(38)
normal test done
testing dealloc_sem
pid=39 is going to wait on sem 0
pid=39 wait on sem 0: success
pid=40 is going to wapid=41 is going to wait on sem 0
pid=42 is going it oto wait on sem 0
n sem 0
parent leaving
$ |

```

```
pid=42 is going to wait on sem 0
in sem 0
parent leaving
$ msgtest

child(sender,child+3): 63(43,66) 44(43,47) 45(43,48) 46(43,49) 47(43,50) 48(43,51) 49(43,52) 50(43,
53) 51(43,54) 52(43,55) 53(43,56) 54(43,57) 55(43,58) 56(43,59) 57(43,60) 58(43,61) 59(43,62) 60(43,
63) 61(43,64) 62(43,65) 63(43,66) 64(43,67) 65(43,68) 66(43,69) 67(43,70) 68(43,71) 69(43,72) 70(43,
73) 71(43,74) 72(43,75) 73(43,76) 74(43,77) 75(43,78) 76(43,79) 77(43,80) 78(43,81) 79(43,82) 80(43,
83) 81(43,84) 82(43,85) 103(43,106) 84(43,87) 85(43,88) 86(43,89) 87(43,90) 88(43,91) 89(43,92) 90(4
3,93) 91(43,94) 92(43,95) 93(43,96) 94(43,97) 95(43,98) 96(43,99) 97(43,100) 98(43,101) 99(43,102) 1
00(43,103) 101(43,104) 102(43,105) 123(43,126) 104(43,107) 105(43,108) 106(43,109) 107(43,110) 108(4
3,111) 109(43,112) 110(43,113) 111(43,114) 112(43,115) 113(43,116) 114(43,117) 115(43,118) 116(43,11
9) 117(43,120) 118(43,121) 119(43,122) 120(43,123) 121(43,124) 122(43,125) 143124(43,127) (43,146) 1
25(43,128) 126(43,129) 127(43,130) 128(43,131) 129(43,132) 130(43,133) 131(43,134) 132(43,135) 133(4
3,136) 134(43,137) 135(43,138) 136(43,139) 137(43,140) 138(43,141) 139(43,142) 140(43,143) 141(43,14
4) 142(43,145) 163(43,166) 144(43,147) 145(43,148) 146(43,149) 147(43,150) 148(43,151) 149(43,152) 1
50(43,153) 151(43,154) 152(43,155) 153(43,156) 154(43,157) 155(43,158) 156(43,159) 157(43,160) 158(4
3,161) 159(43,162) 160(43,163) 161(43,164) 162(43,165)
messages received from :164 165 166 167 168 169 170 171 172 173 174 175 176 177 178 179 180 181 182
183
Hello World! Good Morning!$ semtest
illegal input test: success!
alloc-dealloc test: success!
created semaphores with indices 0 1 2
testing semaphore 0 with initial value 1
-186--186--186--186--186--186--186--186--186--186--186--186--186--186--186--186--186--186--186--186--
leaving(186)
-187--187--187--187--187--187--187--187--187--187--187--187--187--187--187--187--187--187--187--187--
leaving(187)
-188--188--188--188--188--188--188--188--188--188--188--188--188--188--188--188--188--188--188--188--
leaving(188)
-189--189--189--189--189--189--189--189--189--189--189--189--189--189--189--189--189--189--189--189--
leaving(189)
-190--190--190--190--190--190--190--190--190--190--190--190--190--190--190--190--190--190--190--190--
leaving(190)
testing semaphore 1 with initial value 2
-191--192--191--192--191--192--191--192--191--192--191--192--191--192--191--192--191--192--191--192--
-191--192--191--192--191--192--191--192--191--192--191--192--191--192--191--192--191--192--191--192--
leaving(191)

leaving(192)
-193--194--193--194--193--194--193--194--193--194--193--194--193--194--193--194--193--194--193--194--
-193--194--193--194--193--194--193--194--193--194--193--194--193--194--193--194--193--194--193--194--
leaving(193)
-195-
leaving(194)
-195--195--195--195--195--195--195--195--195--195--195--195--195--195--195--195--195--195--195--195-
leaving(195)
testing semaphore 2 with initial value 3
-196--197--198--196--197--198--196--197--198--196--197--198--196--197--198--196--197--198--196--197-
-198--196--197--198--196--197--198--196--197--198--196--197--198--196--197--198--196--197--198--196-
-197--198--196--197--198--196--197--198--196--197--198--196--197--198--196--197--198--196--197--198-
leaving(196)

leaving(197)

leaving(198)
-199--200--199--200--199--200--199--200--199--200--199--200--199--200--199--200--199--200--199--200-
-199--200--199--200--199--200--199--200--199--200--199--200--199--200--199--200--199--200--199--200-
leaving(199)

leaving(200)
normal test done
testing dealloc_sem
pid=201 is going to wait on sem 0
pid=201 wait on sem 0pid=202 is going to wait on sem 0
pid=203 is going to wait on sem 0
pid=204 is going to wait on sem 0
: success
parent leaving
$ |
```

[参考资料]

- [1] XV6 文档 <https://th0ar.gitbooks.io/xv6-chinese/content/content/preface.html>
- [2] Abraham Silberschatz, Peter B. Galvin, Greg Gagne (著), 郑扣根, 唐杰, 李善平 (译). 操作系统概念[M]. 北京机械工业出版社, 2019: 175 ~ 184.

proj4报告

笔记本: OS

创建时间: 2020/6/26 16:27

更新时间: 2020/6/27 16:09

作者: 1300204274@qq.com

proj4 写时复制

邓梟 011750230 1617001

系统启动后会显示空闲页框数，如下

```
xv6...  
cpu0: starting 0  
number of free frames: 56932  
sb: size 1000 nblocks 941 ninodes 200 nlog 30 logstart 2 inodestart 32 bmap  
start 58  
init: starting sh
```

你在内核中添加下面的功能可能会让这个数字变小。

1、写时复制的fork

1.1 任务介绍与提示

fork产生的子进程和父进程有很大的相似性：代码段一样，数据段一样，栈段一样，堆段也一样。xv6在实现fork时，会为子进程的代码、数据、堆、栈均分配物理内存，这有一些浪费，尤其是，若子进程接着调用了exec，则刚分配的物理内存又需要释放。你需要完成以下任务（请参阅教材9.3节的内容）。

- fork执行时，只为子进程创建页表，这个页表完全复制了父进程的页表（包括页框号）。同时，将子进程和父进程的可写页面标记为只读，这些页面称为写时复制页面，它们本来可以被写，只是因为写时复制技术才标记为只读，你需要将这些页面与本来就是只读的页面做区分（缺页中断时需要区分处理）。可以通过页表项中的第9-

11位来做标记（这几位硬件未使用，是供操作系统用的，参见

<http://wiki.osdev.org/Paging>, xv6并没使用这几位)

- 当父进程（或子进程）试图写这些页面时，会触发缺页中断，核实原因为写内存（通过error code判断）并且所写页面为写时复制页面时，再为其分配一个页框，并将原有的内容复制到此页框中，然后修改当前进程的页表项为可写。提示：将页表项的权限改为只读`*pte &= ~PTE_W`；将其设置为可写`*pte |= PTE_W`。
- 当进程消亡时，需要释放页框。但如果两个进程共享一个页框，则提前消亡的进程在释放页框后会造成存活进程无法正常运行。一种策略是为每个已分配的页框记录一个引用数，记录该页框被几个进程的页表所指，每次释放会将引用数减1，当引用数为0时再真正释放页框。你需要在哪些地方维护这个引用数？请写到课设报告中。
- xv6最多支持64个进程，所以引用数用1个字节表示即可（c语言中可用`char`类型）。Makefile中指明物理内存大小为256M（QEMUOPTS中的`-m 256`），据此计算出页框总数，从而得出需要多少字节来存储引用数。为简单起见，你可以在内核里定义一个全局的数组来记录系统启动后页框被进程引用的数量（被内核引用多少次倒不必记录，这里记录的引用数以能满足copy-on-write fork的要求为准）
- 如果你要修改`kfree`，请确保已有代码中的`free_frame_cnt++`和修改空闲链表的代码一起执行，即，二者要么都不执行，要么都执行，否则`free_frame_cnt`的值不准。由于一个页框可能分配给两个进程，所以有可能出现两个进程同时释放页框的情况，即，在对页框的引用数做修改时可能出现竞争条件。复用`kmem.lock`来解决这个问题。

1.2 测试程序

运行`stresstest`，你看到的post的数字应当比下面的数字大(越大越好)。

```
$ stresstest
created 61 child processes
pre: 56785, post: 52271
```

pre和post都可能改表，我们只关心post。比如，一个可能的输出为

```
$ stresstest
created 61 child processes
pre: 56753, post: 52544
```

运行`forktest`，预期输出如下：

```
$ forktest
fork test
fork test OK
```

1.3 准备工作

1.3.1 在本地虚拟机上安装qemu

登录服务器，无操作情况下过几分钟会自动掉线，得重新登录。所以，本次实验，我在WIN10上装一个Linux虚拟机，然后再装qemu。发现不算麻烦，Linux虚拟机用Vmware，然后在linux以下语句即可完成qemu的安装。

```
sudo apt-get install qemu
```

1.3.2 xv6内存管理

xv6的分页机制如下图所示：

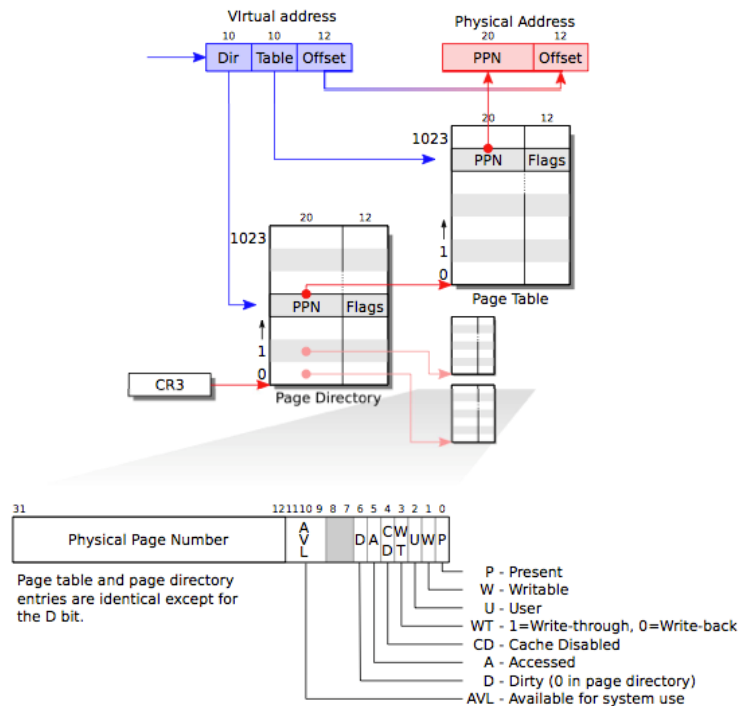


Figure 2-1. x86 page table hardware.

- 内存地址转换单元MMU通过查找进程对应的页表来将逻辑地址转换成物理地址，CR3里存的是项目录的物理基址，通过改变CR3寄存器的值就能改变每个进程使用的页表，即指向进程自己的物理地址空间。xv6的进程项目录基址存在PCB的pgdir中。
- 页表由一级项目录和二级页表项组成，每个项目录占一个页框（4KB），每个项目录项占4B，所以有1024个项目录项。每个项目录项对应一个页表（4KB），每个页表项也是4B，所以每个页表有1024个页表项。
- 由于xv6系统地址线只有32位，所以最高支持4G的地址空间，正好是 $1024 \times 1024 \times 4\text{KB} = 4\text{GB}$ ，所以每个进程的页目录只占一个页框。

- 每个页目录项和页表项由下一级的物理地址（只有页框号）和一些相关的标志位组成，如上图所示。通过设置相应的标志位可实现内存保护等功能。

xv6的进程逻辑地址和物理地址的空间映射如下图所示：

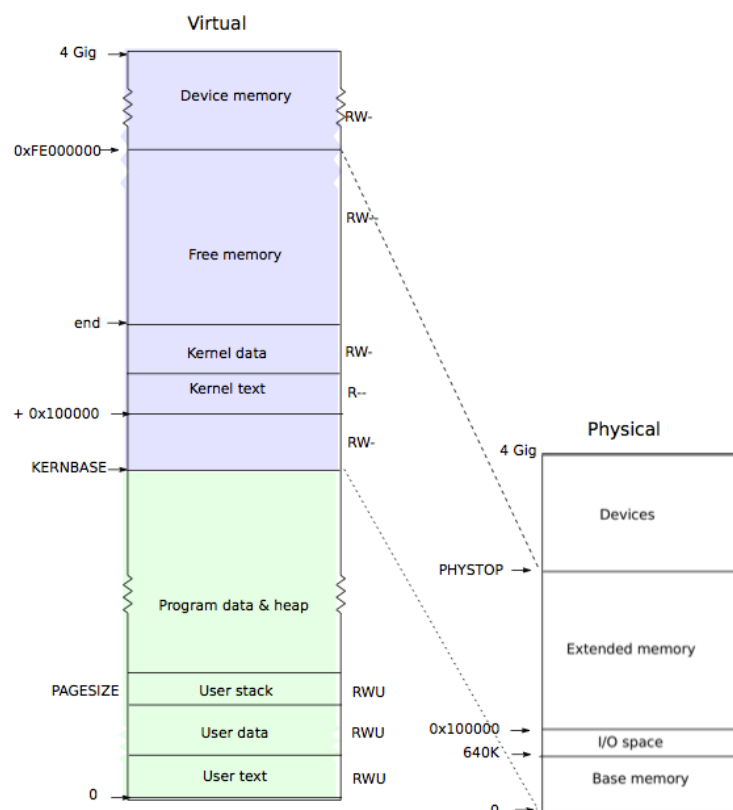


Figure 2-2. Layout of a virtual address space and the physical address space.

详细解释解决方案之前，有必要先了解以下这些管理内存的关键函数的作用，虽不直接使用它们，但有助于理解xv6怎么管理内存。如下：

- `kinit1` 初始化内核末尾到物理内存4M的物理内存空间为未使用
- `kinit2` 初始化 4MB 到PHYSTOP为未使用
- `main` 函数通过调用`kvmalloc` 函数来实现内核新页表的初始化
- `kalloc` 返回虚拟地址空间的地址，`kfree` 以虚拟地址为参数，通过`kalloc` 和`kfree` 能够有效管理物理内存，让上层只需要考虑虚拟地址空间。
- `freerange(void *vstart, void *vend)` 通过调用`kfree` 回收地址空间内的所有页框
- `kalloc (void)`分配出去一个页框
- `switchkvm(void)` Switch h/w page table register to the kernel-only page table,
- `switchvm(struct proc *p)` 把CR3切换到某个进程的页目录基址
- `setupkvm` 通过`kalloc`分配一页内存作为页目录，然后将按照`kmap`数据结构映射内核虚拟地址空间到物理地址空间
- `allocvm` 在设置页表的同时还会分配物理内存(页框)
- `deallocvm` 则相反，它将`newsz`到`oldsz`对应的虚拟地址空间内存置为空闲
- 调用`freevm` 清除用户进程相关的内存环境，`freevm` 首先调用`deallocvm` 将0到KERNBASE的虚拟地址空间回收，然后销毁整个进程的页表
- `copyvm` 负责复制一个新的页表并分配新的内存，新的内存布局和旧的完全一样，xv6使用这个函数作为`fork` 的底层实现

- `mappages` 做的工作是在页表中建立一段虚拟内存到一段物理内存的映射
`mappages(pde_t *pgdir, void *va, uint size, uint pa, int perm)` 调用`walkpgdir`来找到该地址对应的PTE地址
- `walkpgdir` 模仿x86的分页硬件为一个虚拟地址寻找PTE的过程

在`kmalloc.c`中可以看到，`xv6`使用链表管理未分配的页框，并且链表结构体就存在空闲页框的前面（页框内部的前面），内核只需要通过`kmem`保存虚拟地址空间上的`freelist`地址，就能在需要分配页框时找到下一个空闲页框。

`kfree`是释放页框后形成新的节点放到`kmem`的链表`freelist`中；`kalloc`则相反，从`freelist`首部拿走一个空闲页框分配出去。（下面的解决方案并不与此直接相关，即不用直接操作这些结构）

```
int free_frame_cnt=0;
struct run {
    struct run *next;
};

struct {
    struct spinlock lock;
    int use_lock;
    struct run *freelist;
} kmem;
```

1.4 解决方案

1.4.1 扩展kmem的作用域

`kmem` 原来只在`kmalloc.c` 中定义了一个，现在在`vm.c` 中也想使用它来消除对页框的引用数做修改时可能出现竞争条件，那么需要扩展`kmem` 的作用域至少到`vm.c` 中。首先把定义`kmem` 结构体的代码放到`spinlock.h` 中，在`spinlock.h` 中添加代码：

```
//spinlock.h
struct KMEM {
    struct spinlock lock;
    int use_lock;
    struct run* freelist;
};
```

然后在`kalloc.c` 中定义本来就有的变量`kmem`：

```
//kalloc.c
struct KMEM kmem;
```

然后在 vm.c 中添加语句：

```
//vm.c
extern struct KMEM kmem;
```

这样一来，`kmem` 的作用域就扩展到了 `vm.c`，可以用来消除对页框的引用数做修改时可能出现竞争条件。

1.4.2 记录页框被引用的次数

总共有 `totalpg` 个页框，它等于物理内存大小除以页框的大小。页框大小是 4KB，即 2^{12} ，所以将物理内存大小 $256\text{MB} = 2^{28}$ ，右移 12 位就是页框总数。

```
//main.c
#define totalpg (PHYSTOP>>PGSHIFT)
```

xv6 按现在的设定，最多支持 64 个进程，所以每个页框的最大引用数只需要使用一个字节表示，即 `char` 类型已够用。在 `main.c` 中定义全局数组 `page_ref_cnt[totalpg]`，由于是全局的，它会被自动初始化成 0：

```
//main.c
char page_ref_cnt[totalpg];
```

然后在 `vm.c` 中引用此计数数组：

```
extern char page_ref_cnt[];
```

1.4.3 增加页框的被引用次数

`mappages` 的工作是在页表中建立一段虚拟内存到一段物理内存的映射，即它让页表项指向一个页框，在使用 `mappages` 的地方会改变页框被引用的次数。但页框被内核引用多少次我们并不关注，所以在某些使用 `mappages` 的地方我们可以不修改页框被引用的次数，比如 `initvm()` 和 `setupkvm()` 内，因为这里边的 `mappages` 都是映射内核部分的页表项到内核的页框。

纵观 xv6 中引用 `mappages` 的地方就 4 个地方，其中 2 个是上面提到的 `initvm()` 和

`setupkvm()`，另外2个是`allocuvm()`和`copyuvm()`。后面两个函数里使用`mappages`的地方就是页框被用户地址空间引用，所以需要增加页框被引用次数。

allocuvm

首先在`allocuvm`中，其功能是增加进程的用户空间，其页框是通过`kalloc()`刚刚申请到的原来空闲的页框。只要其中的`mappages`执行成功，就增加该页框的被引用次数。修改被引用数的代码如下：

```
/*-----add code -----*/
    acquire(&kmem.lock);
    page_ref_cnt[V2P(mem)>>PGSHIFT]++;
    release(&kmem.lock);
/*-----*/
```

修改之前要拿`kmem.lock`，防止多个进程同时对它进程修改。先用V2P把虚拟地址转换成物理地址，然后除以页框大小得到该页框对应的记录地址，对它“增加1”完成修改后放锁。如果`mappages`执行失败，则不改变该页框的被引用数，而`mappages`执行失败之前引用数被修改过的页框也会在其执行的`deallocuvm`中得到还原。

修改后完整的`allocuvm()`如下：

```
int allocuvm(pde_t * pgdir, uint oldsz, uint newsz)
{
    char* mem;
    uint a;

    if (newsz >= KERNBASE)
        return 0;
    if (newsz < oldsz)
        return oldsz;

    a = PGROUNDUP(oldsz);
    for (; a < newsz; a += PGSIZE) {
        mem = kalloc();
        if (mem == 0) {
            cprintf("allocuvm out of memory\n");
            deallocuvm(pgdir, newsz, oldsz);
            return 0;
        }
        memset(mem, 0, PGSIZE);
        if (mappages(pgdir, (char*)a, PGSIZE, V2P(mem), PTE_W |
PTE_U) < 0) {
            cprintf("allocuvm out of memory (2)\n");
```

```

        deallocvm(pgdir, newsz, oldsz);
        kfree(mem);
        return 0;
    }
    /*-----add code -----*/
    acquire(&kmem.lock);
    page_ref_cnt[V2P(mem) >> PGSHIFT]++;
    release(&kmem.lock);
    /*-----*/
}
return newsz;
}

```

copyvm

在`copyvm`里要做的工作就是把子进程指向其用户空间的页表项先指向其父进程的对应页框，并修改某些标志位，以支持`fork`的写时复制。

以下是修改后的`copyvm()`，下面再一一解释它在干什么。

```

pde_t* copyvm(pde_t * pgdir, uint sz) //传进来了父进程的页目录基址
{
    pde_t* d;
    pte_t* pte;
    uint pa, i, flags;
    //char *mem;

    if ((d = setupkvm()) == 0) //页目录, 按固定方式只映射了内核部分
        return 0;
    for (i = 0; i < sz; i += PGSIZE) {
        if ((pte = walkpgdir(pgdir, (void*)i, 0)) == 0) //为i这个虚拟地址找到了它父进程的pte
            panic("copyvm: pte should exist");
        if (!(*pte & PTE_P))
            panic("copyvm: page not present");
        pa = PTE_ADDR(*pte); //把pte的后12位置0, 即得到实际页框的物理地址

        int notOnlyRead = *pte & PTE_W; // 非0时可写, 否则只读
        if (notOnlyRead) { //可写
            //*pte = *pte & 0xFFFFFFF0;
            *pte &= ~PTE_W;
            *pte = *pte | COWOW; //把父进程的pte标记为COW(写时复制), 并复位可写位(置0)
        }
    }
}

```

```

        else {
            *pte = *pte | COWNW; // 原来不可写
        }
        flags = PTE_FLAGS(*pte); // 留下了 flags
        // if((mem = kalloc()) == 0) // 为子进程申请了一个页框
        // goto bad;
        // memmove(mem, (char*)P2V(pa), PGSIZE); // 复制父进程的页框内容到子进程的页框中
        if (mappages(d, (void*)i, PGSIZE, pa, flags) < 0) // 设置子进程的页表指向父进程相应页框
            goto bad;
        acquire(&kmem.lock);
        page_ref_cnt[pa >> PGSHIFT]++;
        release(&kmem.lock);
    }
    lcr3(V2P(pgdir)); // 不可少
    return d;
bad:
    freevm(d);
    lcr3(V2P(pgdir)); // 不可少
    return 0;
}

```

`setupkvm()` 给子进程申请了一个页框，用作页目录，其中映射到内核的部分已经在 `setupkvm` 中完成了。余下的工作是完成用户空间的写时复制。以下的解释分别针对于每一个页面。

for 循环里干的是，为子进程用户空间的每个页面：

- 找到其父进程相对应页面的页表项 (*pte)，把 *pte 的后12位置0，得到该页面对应的物理地址 `pa`。
- `int notOnlyRead = *pte & PTE_W`；得到该页面原来是否可写。
- 如果父进程的页面原来是可写的，则先用 `*pte &= ~PTE_W`；把它设置成不可写，然后再用 `*pte = *pte | COWOW`；把它标记为原来可写的写时复制页面；
否则用 `*pte = *pte | COWNW`；把它标记为原来不可写的页面。（考虑到页面的可读写性可能会被修改（起码原则上可以），所以作此区分）
宏定义 `COWOW` 和 `COWNW` 可参阅代码，总之它们的作用如上所述，并和缺页处理程序中的使用相对应。
- `flags = PTE_FLAGS(*pte)`；则是把页表项的前20位置0，即留下了后面的标志位。
- 然后语句 `mappages(d, (void*)i, PGSIZE, pa, flags)` 把子进程相应的页表项映射到了物理地址 `pa`，即完成了和父进程共享该页框。
- 随后增加该页框的被引用数。
- 完成页表项的设置后，刷新 `CR3`，以使 `TLB` 的内容与页表内容保持一致。
- 可见中间被注释掉的部分是原来不使用写时复制时的代码，它直接申请一个页框，把父进程相应的内容复制进去。

还有一个地方需要增加页框的被引用数，就是缺页处理程序中新申请页框时，这部分留到提及缺页处理再解释。

1.4.4 减少页框的被引用数

需要减少页框被引用数的地方只有两处，第一处是`deallocvm`，它释放进程的用户空间占用的页框；第二处是缺页处理时，它同样被留到后面再解释。

`deallocvm`

```
int deallocvm(pde_t * pgdir, uint oldsz, uint newsz)
{
    pte_t* pte;
    uint a, pa;

    if (newsz >= oldsz)
        return oldsz;

    a = PGROUNDUP(newsz);
    for (; a < oldsz; a += PGSIZE) {
        pte = walkpgdir(pgdir, (char*)a, 0); //模仿x86的分页硬件为一个虚拟地址寻找PTE的过程
        if (!pte)
            a = PGADDR(PDX(a) + 1, 0, 0) - PGSIZE;
        else if ((*pte & PTE_P) != 0) {
            pa = PTE_ADDR(*pte);
            if (pa == 0)
                panic("kfree");
            /*-----add code-----*/
            acquire(&kmem.lock);
            page_ref_cnt[pa >> PGSHIFT]--;
            release(&kmem.lock);
            if (page_ref_cnt[pa >> PGSHIFT] == 0) {
                char* v = P2V(pa);
                kfree(v);
                *pte = 0;
            }
            /*-----*/
        }
    }
}
```

```
    return newsz;
}
```

对于每一个要被释放的页面对应的页框，先将该页框的被引用数减一，然后判断是否还有某个进程在引用它，如果已没有任何进程在引用它，才调用`kfree` 释放该页框（传入虚拟地址），否则什么也不做。

1.4.5 缺页中断处理程序

查阅资料和试错法一致表明，发生缺页时该进程的PCB的`tf->trapno = 14`。

因为不知道各个`error_code`分别代表什么，所以干脆在缺页处理程序中做了一些判断，只处理对写时复制页面进行写入的情况，其他出错情况只是把进程杀死。

那么在trap中捕获该错误，调用缺页处理程序去处理缺页错误。

```
// trap.c
.....
switch(tf->trapno) {
    case 14:
        page_fault();
        break;
    .....
}
```

中断缺页处理程序如下所示，它被放在`vm.c`中：

```
void page_fault() {
    uint err_addr = rcr2(); //出错的地址
    pte_t* pte;
    uint pa;
    struct proc* curproc = myproc();
    //cprintf("%d into page_fault\n", curproc->pid);
    if (err_addr >= KERNBASE) {
        curproc->killed = 1;
        return;
    }
    else if ((pte = walkpgdir(curproc->pgdir, (void*)err_addr, 0)) == 0)
    { //页表项不存在
        curproc->killed = 1;
        return;
    }
    else if (!(*pte & PTE_P)) { //页框不present
        curproc->killed = 1;
        return;
    }
}
```

```

    }
    else if (!(*pte & PTE_U)) {//错误访问内核
        curproc->killed = 1;
        return;
    }
    else if ((*pte & 0x00000600) == 0x200) {//对不可写页进行写操作
        curproc->killed = 1;
        return;
    }
    else {//写时复制
        pa = PTE_ADDR(*pte);
        acquire(&kmem.lock);//保证下面的读时没有其他线程在改变它
        if (page_ref_cnt[pa >> PGSHIFT] == 1) {//如果此时只有一个页
还在引用它，直接标为可写即可，并清除写时复制标记
            *pte = *pte | PTE_W;
            *pte = *pte & 0xFFFFF1FF;//清除写时复制标记
            release(&kmem.lock);
        }
        else {
            char* mem;
            release(&kmem.lock);// kalloc里也要拿锁
            if ((mem = kalloc()) == 0) {
                cprintf("Alloc for cow wrong\n");
                curproc->killed = 1;
                return;
            }
            int flags = PTE_FLAGS(*pte);//留下了flags
            flags &= 0xFFFFF1FF;
            flags |= PTE_W;
            memmove(mem, (char*)P2V(pa), PGSIZE);
            acquire(&kmem.lock);
            page_ref_cnt[pa >> PGSHIFT]--;
            page_ref_cnt[V2P(mem) >> PGSHIFT]++;
            release(&kmem.lock);
            *pte = V2P(mem) | flags;
        }
        lcr3(V2P(curproc->pgdir));
    }
}

```

- 使用 `rcr2()` 从 CR2 中读出导致发生缺页中断的逻辑地址（虚拟地址或称线性地址），经过一系列的错误判断（如注释所示），如果不是对写时复制页面进行写操作，则杀死进程；否则对缺页错误进行处理。

- 通过 `pa = PTE_ADDR(*pte)` 拿到想写的页框物理地址，记得先拿 `kmem.lock` 再判断页框的被引用数，
 - ①如果该页框现在的被引用数是1，则不用分配新页框，直接把该页框改为可写并清除写时复制标记。
 - ②否则，分配新页框给该进程。将该页面的内容复制入新分配到的页框，修改原页框和新分配页框的被引用数。注意，新分配的页框不是写时复制。
- 最后记得 `lcr3(V2P(curproc->pgdir))`；刷新TLB。
- 调用 `kalloc()` 之前要先 `release(&kmem.lock)`；因为 `kalloc()` 里本来也要 `acquire(&kmem.lock)`。

2、开放实验

实际系统中还有其他可以节省物理内存的技术。当你实现这些技术后，你应当依然能够正常通过 `forktest`。此时，运行 `stresstest`，`post` 的数字越大越好。

- 一种节省内存的方法是，减少内核页表的复制，即每个进程在创建时，其实映射到内核空间的页表可以复用，不需要创建这部分页表。

2.1 减少内核页表的复制

再看一下这张图：

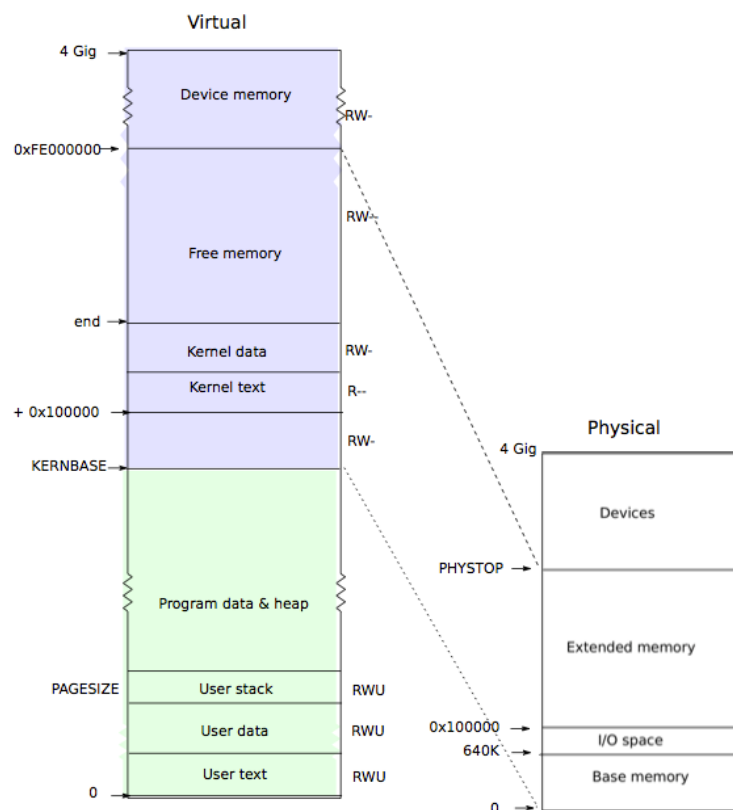


Figure 2-2. Layout of a virtual address space and the physical address space.

(后来的) setupkvm调用都按如下kmap[]进行内核逻辑地址的映射：

```
kmap[] = {
    { (void*)KERNBASE, 0,          EXTMEM,      PTE_W}, // I/O
    space
    { (void*)KERNLINK, V2P(KERNLINK), V2P(data), 0},      // kern
    text+rodata
    { (void*)data,      V2P(data),      PHYSTOP,    PTE_W}, // kern
    data+memory
    { (void*)DEVSPACE, DEVSPACE,        0,          PTE_W}, // more
    devices
};
```

- 一个页表可映射的空间大小是 $4 * 1024KB = 2^{22}B$
- 按设置xv6的物理内存是256MB = $2^{28}B$ ，需要 $2^6 = 64$ 个页表，那么至少需要64个页表项来映射内核空间，那么内核空间到底使用了哪些页目录项来映射映射内核空间呢？想要节省映射内核的页表，就让进程们共用那些页目录项对应的页表。

xv6使用setupkvm来映射内核页表，第3次调用setupkvm时的映射是后来每个进程都是用的。至于为什么是第3次，我是经过调参调出来的，原理实在没时间分析了，过程如下：

- 设置变量setupkvm_cnt 来记录系统现在第几次调用setupkvm，初始化成0。

```
pde_t* setupkvm() {
    if (setupkvm_cnt <= x) {
        //功能和原来的setupkvm一样
    } else {
        //执行我的setupkvm
    }
}
```

其中的x 就是我调的参数。最终发现x是2时，即第3次调用setupkvm 设置的页目录的和内核相关的部分是后面创建的每个进程都使用的。

- 用refer_pgdir 记录第3次调用setupkvm 设置的页目录基址，该目录至少在关机之前不会被释放，以后创建新页目录的内核部分的页目录项就复制refer_pgdir 相应的页目录项，它是最先创建的有效页目录，非常核心，这确保了该页目录关机之前不会被销毁。
- $*(pgdir + i) |= 0x00000800$ ；用来标记本页目录项是指向共享内核页表的，结束进程时不能删除该页表，这需要在freevm 中做相应的改动。
- 那怎么知道共享哪些页目录项指向的页表呢？
在xv6的当前设置下，经过计算可以得到是页目录项[512, 568) 和[1016, 1024)。
但为了有更好的适应性，可以在程序中判断并作记录。方法是在第三次调用setupkvm时判断哪些页目录项被使用了，记录它们所在的区间。

修改后的setupkvm() 如下所示：

```

int setupkvm_cnt = 0;
int pgstart1;
pde_t* refer_pgdir;
pde_t* setupkvm(void)
{
    pde_t* pgdir;
    //cprintf("setupkvm_cnt = %d, and the content is\n",
    setupkvm_cnt);
    if (setupkvm_cnt <=2) {
        struct kmap* k;
        if ((pgdir = (pde_t*)kalloc()) == 0)
            return 0;
        memset(pgdir, 0, PGSIZE);
        if (P2V(PHYSTOP) > (void*)DEVSPACE)
            panic("PHYSTOP too high");
        for (k = kmap; k < &kmap[NELEM(kmap)]; k++)
            if (mappages(pgdir, k->virt, k-
>phys_end - k->phys_start,
                                (uint)k->phys_start,
k->perm) < 0) {
                                freevm(pgdir);
                                return 0;
                                }
        pgstart1 = KERNBASE / 0x400000;
        setupkvm_cnt++;
        refer_pgdir = pgdir;
        return pgdir;
    }
    else {
        if ((pgdir = (pde_t*)kalloc()) == 0)
            return 0;
        memset(pgdir, 0, PGSIZE);
        int i;
        for (i = pgstart1; i < 1024; i++) {
            *(pgdir + i) = *(refer_pgdir + i);
            *(pgdir + i) |= 0x00000800; //公用
内核页表, 没映射到页表也无所谓
        }
        return pgdir;
    }
}

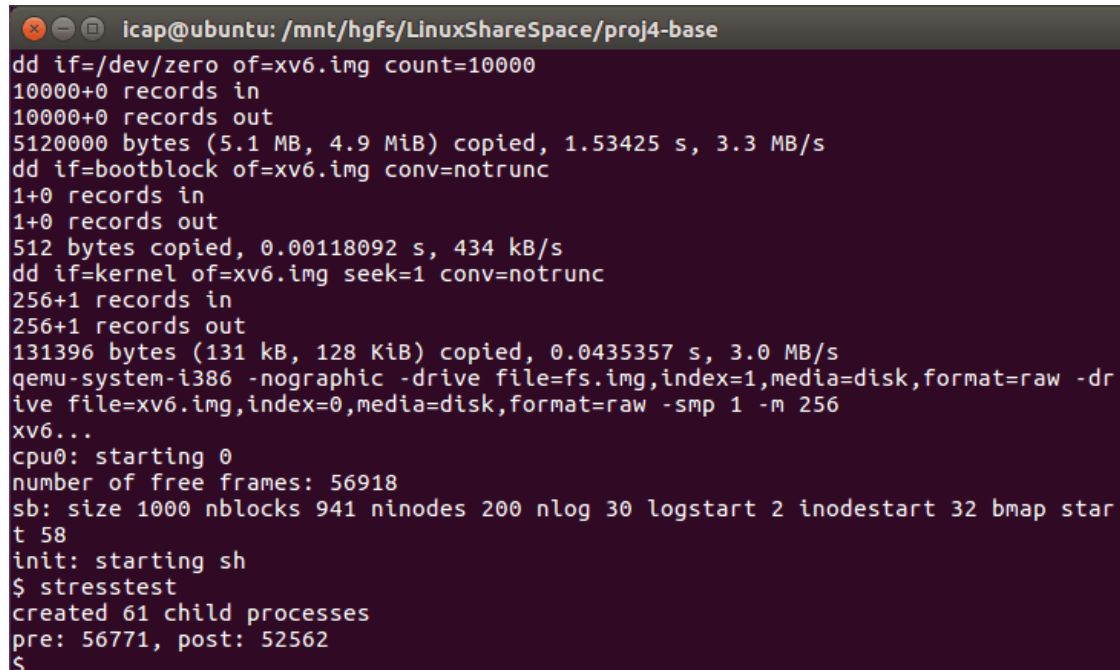
```

那么`freevm`在释放页表的页框时，还要判断此页框是不是存放着公共的内核页表的，如果是则不能释放它。

```
void freevm(pde_t * pgdir)
{
    uint i;
    if (pgdir == 0)
        panic("freevm: no pgdir");
    deallocvm(pgdir, KERNBASE, 0);
    for (i = 0; i < NPENTRIES; i++) {
        int f;
        f = pgdir[i] & 0x00000800; //f 非0则表示该页框是存公共指向内核的页表的
        if ((pgdir[i] & PTE_P) && f==0) {
            char* v = P2V(PTE_ADDR(pgdir[i]));
            kfree(v);
        }
    }
    kfree((char*)pgdir);
}
```

3 实验结果

完成写时复制，但没有完成共享内核页表时：



```
icap@ubuntu: /mnt/hgfs/LinuxShareSpace/proj4-base
dd if=/dev/zero of=xv6.img count=10000
10000+0 records in
10000+0 records out
5120000 bytes (5.1 MB, 4.9 MiB) copied, 1.53425 s, 3.3 MB/s
dd if=bootblock of=xv6.img conv=notrunc
1+0 records in
1+0 records out
512 bytes copied, 0.00118092 s, 434 kB/s
dd if=kernel of=xv6.img seek=1 conv=notrunc
256+1 records in
256+1 records out
131396 bytes (131 kB, 128 KiB) copied, 0.0435357 s, 3.0 MB/s
qemu-system-i386 -nographic -drive file=fs.img,index=1,media=disk,format=raw -drive file=xv6.img,index=0,media=disk,format=raw -smp 1 -m 256
xv6...
cpu0: starting 0
number of free frames: 56918
sb: size 1000 nblocks 941 ninodes 200 nlog 30 logstart 2 inodestart 32 bmap start 58
init: starting sh
$ stresstest
created 61 child processes
pre: 56771, post: 52562
$
```

(以上是我在本地虚拟机上的界面)

(在本地虚拟机上编辑的代码，传到服务器上后，中文注释乱码，不过不影响结果。)

又完成共享内核页表后：

```
dengxiao@VM-199-186-ubuntu: ~/proj4-base
1+0 records in
1+0 records out
512 bytes copied, 0.00119034 s, 430 kB/s
dd if=kernel of=xv6.img seek=1 conv=notrunc
257+1 records in
257+1 records out
131632 bytes (132 kB, 129 KiB) copied, 0.0033394 s, 39.4 MB/s
qemu-system-i386 -nographic -drive file=fs.img,index=1,media=disk,format=raw -drive file=xv6.img,index=0,media=disk,format=raw -smp 1 -m 256
xv6...
cpu0: starting 0
number of free frames: 56918
sb: size 1000 nblocks 941 ninodes 200 nlog 30 logstart 2 inodestart 32 bmap start 58
init: starting sh
$ stresstest
created 61 child processes
pre: 56898, post: 56593
$ forktest
fork test
fork test OK
$ stresstest
created 61 child processes
pre: 56898, post: 56593
$
```

综上：post = 56593

共享内核页表的原理：每个进程在创建时，其实映射到内核空间的页表可以复用，不需要每次都创建这部分页表，且这些内核页表创建之后在关机之前都不释放。

[参考资料]

[1] XV6 文档 <https://th0ar.gitbooks.io/xv6-chinese/content/content/preface.html>

[2] xv6源码分析(四):内存管理

https://blog.csdn.net/qq_25426415/article/details/54633843